

基于单元格扩展的报表引擎的研究与实现

客服

分享

收藏

立即下载 会员免费下载

原视图版网页已模拟裁切为如下版式

基于单元格扩展的报表引擎的研究与实现

开通豆丁会员，低至三折下载文档

开通会员

基于单元格扩展的报表...实现.pdf

8.68M82页 5阅读zhhj000012019-12-02

立即下载 举报

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

摘要

所谓报表，就是用表格、图表对数据进行展示的一种媒介，报表的使用在企业无处不在。优秀的报表可以对基础数据进行过滤、合并、统计、分析，提供直观的达形式，并为决策者提供良好的依据。

本文首先研究对比了国内外报表工具的优缺点，包括 BIRT、JasperReport、Pentaho 和润乾报表等。中国式报表又有几个重要的特征：多数据源、分片、跨行计算，这些报表工具在使用场景、设计方式等方面都与中国式报表有出入。然后，本摒弃了传统报表工具的条带式设计方式，借鉴了 EXCEL 的单元格设计，提出了一种于单元格扩展的报表引擎实现方法。在对 EXCEL 的借鉴上，采用了基于单元格的方式，采用基于单元格的命名方式，采用基于单元格的基本属性设置方式，采用基单元格的内容表达方式。基于单元格的扩展方法以单元格之间的继承关系为核心，现数据的过滤、合并，从而得到描述结果报表的最终结果矩阵。其次，本文设计了一种针对最终结果矩阵的分页方法，该方法适用于普通分页情况和有重复元素的分页况。最后，根据中国式报表特征设计多种报表应用场景，验证该报表引擎的展示及出情况，并对测试结果进行分析总结。

测试表明，基于单元格扩展的报表引擎可以有效的满足中国式报表的需求。

关键字：中国式报表；多数据源；XML 模板；单元格扩展

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

Abstract

A report is a medium for presenting data with tables and charts, and the use of reports is ubiquitous in the enterprise. Excellent reports can filter, merge, statistics, and analyze the data, provide a visual representation and a good basis for decision makers.

Firstly, this article looked at the pros and cons of reporting tools at home and abroad, including BIRT, JasperReport, Pentaho and other reports. There are several important features of Chinese reports: multiple data sources, segmentation, cross group computing, it shows that these tools have problems in designing and representing Chinese reports. Secondly, this article abandoned the traditional strip-type design method, adopted a design method based on cell expansion and implemented a report engine based on cell expansion theory. This theory learned from EXCEL, including the naming way of cell unit, the basic properties setting way of cell unit and the contents expression of cell unit. This theory realized data filtration and merge by using inheritance relationships between cells as core. Thirdly, this article designed a paging way to get the final result matrix which is used in normal paging situation and repeat paging situation, described the representation way of final result. Finally, it designed several application scenarios according to the features of Chinese reports, tested the presentation and execution foundation of the report engine, and summarized the test results.

The test results showed that this report engine based on cell expansion can meet the needs of Chinese reports.

Keywords: Chinese Report; Multiple Data Source; XML Template; Cell Expansion



基于单元格扩展的报表引擎的研究与实现



目录

摘要.....	
Abstract.....	
第一章 绪论	
1.1 引言.....	
1.2 国内外研究现状.....	
1.3 主要工作和研究内容.....	
1.4 论文组织结构.....	
第二章 相关技术分析与研究	
2.1 XML 技术	
2.1.1XML 简介	
2.1.2XML 解析技术	
2.2 POI 技术	
2.3 Servlet 技术	
第三章 系统整体架构	
3.1 系统整体架构.....	
3.2 报表引擎工作流程.....	
3.3 系统核心类图.....	
3.4 模板描述文件.....	
3.4.1数据源.....	
3.4.2工作表.....	
3.4.3参数列表.....	
3.4.4样式列表.....	
3.5 结果工作簿.....	
3.6 报表引擎优化策略.....	
3.6.1WorkBookEntry.....	
3.6.2ResultCache.....	
3.6.3SessionInf.....	
第四章 单元格扩展原理	
4.1 报表设计.....	
4.2 计算单元格继承关系.....	
4.2.1单元格的继承关系的过滤作用.....	
4.2.2计算单元格的继承关系.....	

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

4.2.3 继承关系矩阵组成.....	
4.2.4 计算继承关系矩阵.....	
4.3 初始化表格矩阵.....	
4.3.1 表格矩阵组成.....	
4.3.2 初始化表格矩阵.....	
4.4 填充表格矩阵.....	
4.5 扩展子格、更新继承关系.....	
4.5.1 扩展子格.....	
4.5.2 更新继承关系.....	
4.5.3 扩展、更新过程.....	
4.6 计算最终结果矩阵.....	
4.6.1 计算单元格深度序列.....	
4.6.2 单元格定位顺序.....	
4.6.3 最终结果对象: BoxCase.....	
4.6.4 继承区域对象: ExtendedArea.....	
4.6.5 设置数值单元格的最终位置.....	
4.6.6 重新计算映射序列.....	
4.6.7 计算最终结果矩阵.....	
第五章 工作簿分页与数据展示	
5.1 影响分页的因素.....	
5.1.1 纸张可用宽高.....	
5.1.2 单元格大小.....	
5.1.3 重复行列.....	
5.1.4 行列分页属性.....	
5.2 获取工作表的行列分页信息.....	
5.3 生成页面分布序列.....	
5.3.1 区域分布序列.....	
5.3.2 普通分页.....	
5.3.3 有重复元素的分页.....	
5.4 生成分页序列.....	
5.5 数据展示.....	
5.5.1 HTML 页面展示	
5.5.2 EXCEL 文档展示	
第六章 测试结果及结果分析	
6.1 多源报表.....	



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

- 6.3 分组报表.....
- 6.4 交叉报表.....
- 6.5 按组分页.....
- 6.6 导出 Excel 文档.....
- 第七章 总结与展望
- 参考文献.....
- 攻读硕士学位期间取得的研究成果.....
- 致谢.....





第一章 绪论

1.1 引言

报表是数据信息的载体，在企业中，报表的使用无处不在^[1]。通过对企业生产经营活动进行汇总、统计和分析，可以使企业相关人员多层次、多角度、全面详细地了解企业生产状态，从而做出进一步决策。优秀的报表可以对基础数据进行过滤、统计、整合、分析，提供形象直观的表达形式，为决策者提供良好的依据^[2]。报表是企业信息管理系统中不可缺少的重要部分。

所谓报表，就是用表格、图表对数据进行展示的一种媒介^[3]。在企业应用系统中用到的报表，一般有两种类型，一种是由企业开发人员自主开发的，一种是由报表工具自动生成的^[4]。

目前大型企业基于 B/S 的报表系统开发都是开发人员根据实际需求编写代码，而实现数据的过滤、统计、整合、分析、展示功能。企业自主开发的报表模块虽然对性强，但存在编程复杂、重复工作多、难度大、通用性差、很难实现复用等缺点^[5]。

国内外诸如 BIRT^[7]、JasperReport^[8]、润乾报表^[9]等著名的报表工具，有的价格贵，有的入门困难，有的报表设计和使用方面不符合中国式报表的习惯，无法满足企业定制复杂的交叉表、复核表等复杂报表的需求^[10-11]。

中国式报表有四个重要特征：多数据源、跨行组的运算、分片、和填报合一^[12]

1. 多数据源是指同一张报表的数据可能来自多个不同的异构数据库；
2. 跨行组的运算是指报表统计中占比、环比、同期比之类的数据运算；
3. 分片是指同一批数据按不同的主题统计汇总；
4. 填报合一是指报表展示支持数据录入的功能；

因此，实现一个对业务需求进行高层次抽象、数据统一处理、统一展示报表引擎系统很有必要。它既能够缩短报表系统的开发周期、减少重复工作、降低开发成本，能够符合中国式报表的要求。

1.2 国内外研究现状

随着大数据和开放数据经济的兴起，如何对这些数据进行分析并将其以一种可的形式准确展现出来也变得越来越重要。



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

国内外有许多优秀的报表工具，例如 BIRT、JasperReport、Pentaho、润乾报表 [13]。

BIRT 是一个基于 Eclipse 的开源报表系统，它主要用在 Web 应用程序上，它以 JAVA 和 JavaEE 为基础。BIRT 主要由两部分组成：一基于 Eclipse 的报表设计器、一个是部署到应用服务的运行时组件。BIRT 是开源的报表产品，可以降低开发成本操作。同时，它界面友好大方，集成在 Eclipse 中，使用方便。

JasperReport^[14]是一个基于 Java 的开源报表工具，iReport 是一个开源的图形化辅助工具，可以用来进行报表的设计。JasperReport 和 iReport 一起使用，功能强大设计灵活，并且支持 PDF、HTML 等多种格式的文件输出。

Pentaho^[15]是一个基于 Java 平台的商业智能套件。它以工作流为核心、强调面解决方案而非工具组件，整合了多个组件（报表组件、分析组件、图表组件、数据集组件、数据挖掘组件等），侧重于大中型企业应用。

润乾报表^[16]是一个纯 JAVA 的企业级报表工具，支持对 JavaEE 系统的嵌入式部署、无缝集成。它提供了高效的报表设计方案、强大的报表展现能力、灵活的部署制，为企业级数据分析与商业智能提供了高性能、高效率的报表系统解决方案。

传统的报表设计器，例如 BIRT、JasperReport，使用了条带式的设计体验，也将一张报表分成若干个条状区域。以 JasperReport 为例，一个报表的结构，一般分下几个分区：标题、页眉、列表头部信息、报表详细信息区域、列表尾部信息、页脚、总结^[18]。设计报表模板时先确定使用哪些条状区域，然后在这些条状区域内放元素。这样的设计虽然简单，但是格式僵化，缺乏变通^[19]，而且不能满足复杂场景中国式报表的要求。JasperReport 条带式设计器设计器如下图所示：







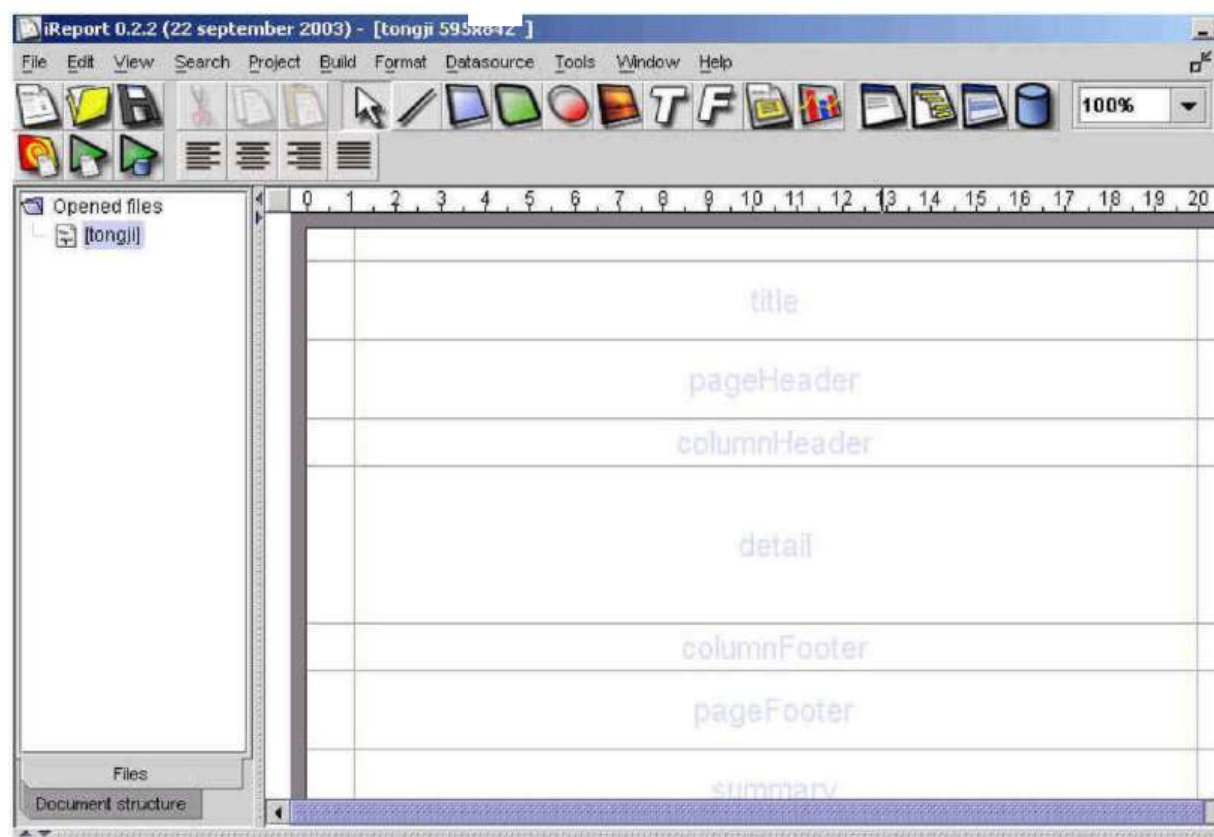


图 1-1 条带式设计器示意图

1.3 主要工作和研究内容

针对现有报表系统存在的问题，本文的报表设计完全摒弃条带式的设计方式，鉴了 Excel 的表现形式，采用了基于单元格的设计方式。报表中的所有元素都是放在单元格中的。本文的报表引擎对 Excel 的借鉴如下：

1. 采用 Excel 中单元格的命名方式，利用单元格在行列上所处的位置来给单元命名，例如 A2、C4 等；
2. 采用 Excel 中单元格的基本属性设置方式，支持单元格中数据的字体、大小、颜色、边框、背景色等基本属性的设置；
3. 采用 Excel 中单元格内容的表达方式，例如，公式单元格的“=SUM(A3+B5)”等；

在基于单元格的设计基础上，本文提出了一种基于单元格的扩展方法。扩展方法以单元格之间的继承关系为核心，实现报表单元格的扩展、填充、过滤、合并最终得到描述报表结果的最终结果矩阵。

而最终结果矩阵很有可能非常庞大，但是一个页面所能呈现的数据是有限的，此需要先对最终结果矩阵进行分页，然后进行分页展示。针对这种情况，本文提出





此外，本文的报表引擎针对中国式报表的特征，具有以下特点：

1. 基于 Java 开发，具有良好的跨平台性；
2. 采用 XML 文件描述报表模板，报表模板设计规则简单；
3. 天然支持多源数据；
4. 自定义一种单元格扩展方法，有效解决了交叉报表、跨行组的运算等复杂报表的需求；
5. 可以导出为 HTML，可以实现在 J2EE 的 Web 报表应用中的集成和部署；
6. 提供报表的 EXCEL 文档的导出功能；

1.4 论文组织结构

本文共有 7 章，具体组织架构如下：

第一章为绪论，介绍报表在企业中的重要性、中国式报表的特点以及国内外报表工具的现状，简略说明论文主要工作和研究内容及论文组织结构。

第二章为相关技术分析与研究，主要包括 XML 技术、POI 技术、Servlet 技术。

第三章为系统整体架构分析，主要介绍了报表引擎的工作流程、报表引擎的核架构、工作簿的构成；

第四章为单元格扩展原理，详细描述了如何从抽象的单元格定义扩展成为最终结果矩阵的过程。

第五章为工作簿分页过程与数据展示，详细讲述了如何对最终结果矩阵进行分页，将结果报表输出为 HTML 数据和 EXCEL 文档的过程。

第六章为报表场景应用，对基于单元格扩展原理的报表引擎的应用场景和生成成果进行展示。

第七章为总结与展望，总结论文主要工作与研究内容，针对报表引擎依旧存在的、尚未解决的问题进行展望。





第二章 相关技术分析与研究

2.1 XML 技术

2.1.1 XML 简介

XML 是一种通用的数据交换格式，它的平台无关性、语言无关性、系统无关性、简单易用性给数据集成与交互带来了极大的便利^[20-21]。

XML 语法简单，它主要包括文档声明、处理指令、标签、属性、注释、CDATA 块 6 个元素^[22]。各元素的作用和语法如下：

1. 文档声明：声明 XML 文档的版本和编码；例如，`<?xml version="1.0" encoding="utf-8"?>`；
2. 处理指令：声明如何解析 XML 文档；例如，`<?xml-stylesheet type="text/css" href="a.css"?>` 声明该 XML 文档引用了 a.css；
3. 标签：由开始标签`<`和结束标签`>`构成，xml 标签名称区分大小写，标签定要正确配对、标签名中间不能使用空格、标签名不能以数字开头；
4. 属性：属性在开始标签中声明，属性值必须以引号包含，一个标签内可以有多个属性但不能出现重复的属性名；
5. 注释：注释语句包含在`<!-- -->`中；
6. CDATA 块：CDATA 块可以让一些需要包含特殊字符的内容统一进行原样输出，例如`<字符`；

XML 广泛应用于配置文件中。例如，在 Tomcat 的初始化配置文件、著名的 SSH^[23]（Struts、Hibernate、Spring）框架的配置文件都可以看到 XML 的身影。同时，XML 也可以作为小型的数据载体。

Hibernate^[24]的配置文件如下所示：



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

```
<!-- 导入外部的properties配置文件 -->
<context:property-placeholder location="classpath:db.properties" />

<!-- 配置c3p0数据源 -->
<bean id="dataSource" class="com.mchange.v2.c3p0.ComboPooledDataSource" destroy-method="close">
    <property name="jdbcUrl" value="${jdbcUrl}"></property>
    <property name="driverClass" value="${driverClass}"></property>
    <property name="user" value="${user}"></property>
    <property name="password" value="${password}"></property>
    <!--初始化时获取三个连接，取值应在minPoolSize与maxPoolSize之间。Default: 3 -->
    <property name="initialPoolSize" value="${initialPoolSize}"></property>
    <!--连接池中保留的最小连接数。Default: 3 -->
    <property name="minPoolSize" value="3"></property>
    <!--连接池中保留的最大连接数。Default: 15 -->
    <property name="maxPoolSize" value="${maxPoolSize}"></property>
    <!--当连接池中的连接耗尽的时候c3p0一次同时获取的连接数。Default: 3 -->
    <property name="acquireIncrement" value="3"></property>
    <!--最大空闲时间，1800秒内未使用则连接被丢弃，若为0则永不丢弃。Default: 0 -->
    <property name="maxIdleTime" value="1800"></property>
</bean>
```

图 2-1 Hibernate 配置文件

2.1.2 XML 解析技术

XML 解析器是 XML 应用的基础。目前有两种主流的 XML 解析方式：DOM 解析和 SAX 解析^[25]。

DOM 解析^[26]会一次性把 XML 文档加载成 Document 树，通过 Document 对象和节点对象，通过节点对象访问 xml 文档内容（标签，属性，文本，注释）。它可以对整个 XML 文档的全面、动态访问。

```
<?xmlVersion="1.0" encoding="utf-8"?>
<contactList>
    <contact id="001">
        <name>张三</name>
        <age>24</age>
        <phone>12345643223</phone>
        <email>zhangsan@qq.com</email>
    </contact>
    <contact id="002">
        <name>李四</name>
        <age>25</age>
        <phone>9876849589</phone>
        <email>lisi@qq.com</email>
    </contact>
</contactList>
```

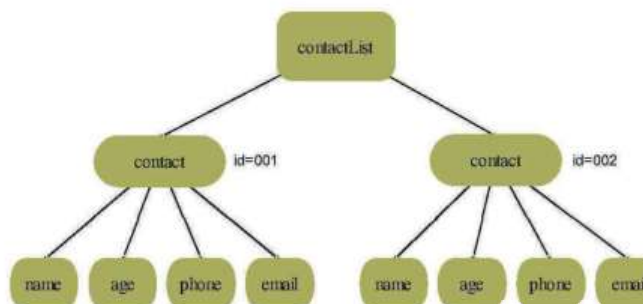


图 2-2 DOM 解析示意图

SAX 解析^[27]提供了一种对 XML 文档进行顺序访问的模式。当使用 SAX 解析器对 XML 文档进行解析时，会触发一系列事件，所以 SAX 接口也被称作事件驱动接口^[28]。

DOM 解析和 SAX 解析的优缺点^[29]如下所示：





XML解析方式	优点	缺点
DOM解析	1. 内存树结构提供了很好的导航功能; 2. DOM解析的应用程序的编写比较简单;	1. 内存消耗大; 2. 运行速度较慢;
SAX解析	1. 内存消耗小, 可以解析任意大小的文件; 2. 适合创建自己的数据结构; 3. 可以做部分解析, 适合销量信息子集;	1. 导航功能弱; 2. SAX解析的应用程序的编写比较复杂;

图 2-3 DOM 解析和 SAX 解析比较

JAXP (Sun 公司)、JDOM 工具和 Dom4J 都是基于 DOM 原理的 XML 解析工具。其中, Dom4J 是三大框架默认读取 XML 的工具。基于 SAX 原理的 XML 解析工具 Sun 公司的 Sax 解析工具。

2.2 POI 技术

很多时候, 一个软件应用程序需要生成 Microsoft Excel 文件格式的报告。有时一个应用程序甚至希望将 Excel 文件作为输入数据。

JAVA 中操作 Excel 的有两种比较主流的工具包: JXL^[30]和 POI^[31]。

JXL 只能操作 Excel 2000 及以前版本的 Excel 文档,而 POI 可以操作 Excel 95 及后的版本, 即可操作后缀为.xls 和.xlsx 两种格式的 Excel 文档。

Apache POI^[32]是一种流行的 API, 它允许程序员使用 Java 程序创建、修改和显示 MS Office 文件。POI 全称 Poor Obfuscation Implementation, 利用 POI 接口可以通过 JAVA 操作 Microsoft Office 套件工具的读写功能。POI 支持 Office 的所有版本。

Apache POI 提供的组件如下:

HSSF	提供读写Microsoft Excel XLS格式档案的功能
XSSF	提供读写Microsoft Excel OOXML XLSX格式档案的功能
HWPf	提供读写Microsoft Word DOC97格式档案的功能
XWPF	提供读写Microsoft Word DOC2003格式档案的功能
HSLF	提供读写Microsoft PowerPoint格式档案的功能
HDGF	提供读Microsoft Visio格式档案的功能
HPBF	提供读Microsoft Publisher格式档案的功能
HSMF	提供读Microsoft Outlook格式档案的功能

图 2-4 POI 组件示意图

2.3 Servlet 技术

Servlet^[33]是在服务器上运行的小程序。它的任务就是根据一个客户端的请求, 而



回一个响应。

Servlet 的生命周期^[34]如下所示：



图 2-5 Servlet 的生命周期

Servlet 的主要生命时刻^[35]如下所示：

表 2-1 Servlet 的主要生命时刻

生命时刻	何时调用	作用
init()	Servlet 实例被创建之后，并在 Servlet 能为客户请求提供服务之前，容器要对 Servlet 调用 init()	在 Servlet 处理客户请求之前对其进行初始化
service()	当第一个客户请求到来时，容器会开启一个新线程，并调用 Servlet 的 service() 方法	查看请求，确定 HTTP 方法（GET、POST 等），并在 Servlet 上调用对应的方法，如 doGet()或 doPost()等
doGet()/doPost()	Service()方法根据请求的 HTTP 方法（GET、POST 等）来调用 doGet()或 doPost()	对应HTTP 请求(GET、POST 等)的具体操作
destroy()	容器关闭之前	杀死 Servlet，垃圾回收



基于单元格扩展的报表引擎的研究与实现



第三章 系统整体架构

3.1 系统整体架构

本文描述的报表系统的整体架构如下所示：

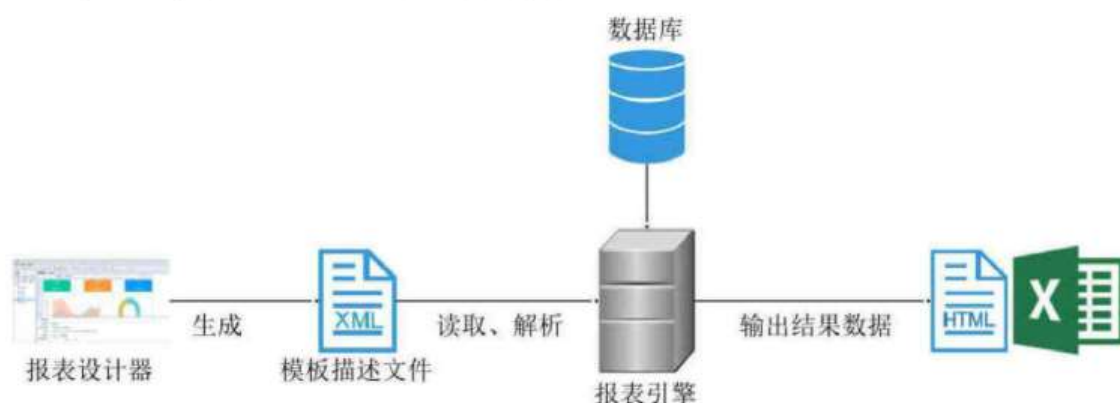


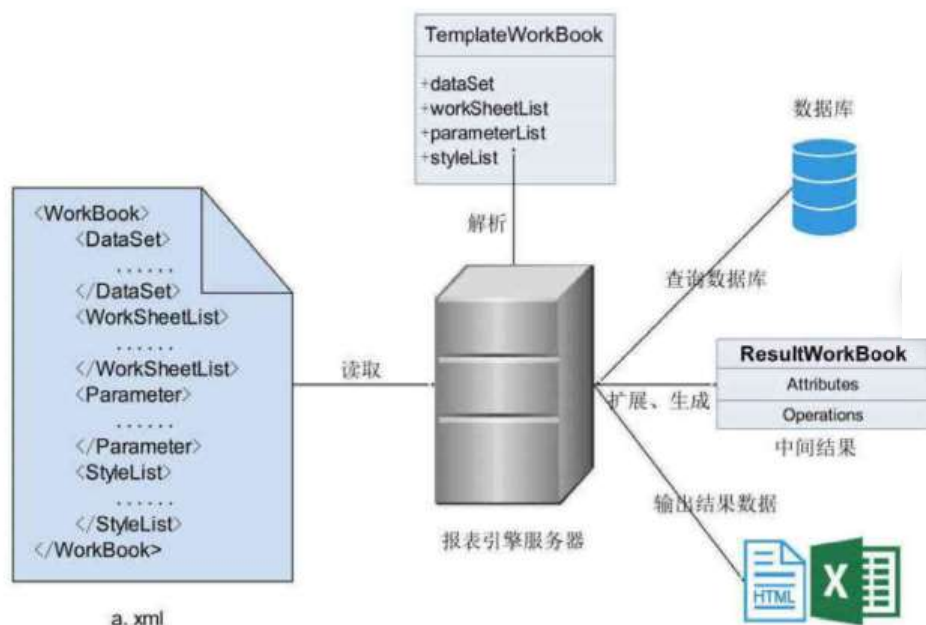
图 3-1 系统整体架构示意图

首先，由报表设计人员根据需求设计报表生成 XML 格式的模板描述文件。然后由报表引擎对模板描述文件进行读取、解析、查询数据库，扩展生成中间结果。最后，报表引擎根据用户需要输出多种格式的结果数据。

本文的工作重点在于模板描述文件的读取、解析过程和最终结果的输出过程，不涉及报表设计器的设计实现。

3.2 报表引擎工作流程

报表引擎的工作就是先读取、解析模板描述文件，然后查询数据库、扩展生成中间结果，最后输出多种格式的结果数据。报表引擎的工作原理如下图所示：







3.3 系统核心类图

该报表引擎的核心类图如下所示：

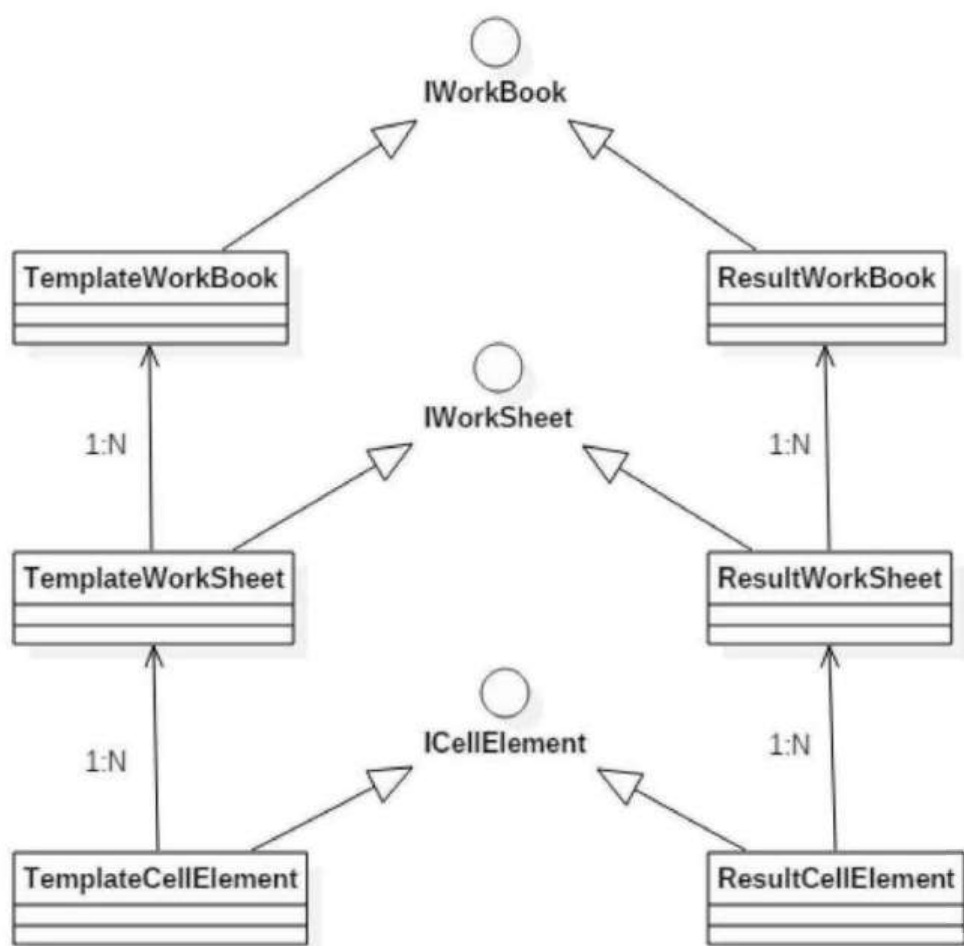


图 3-3 系统核心类图

其中，模板工作簿 TemplateWorkbook 和结果工作簿 ResultWorkbook 都实现了工作簿接口 IWorkbook，模板工作表 TemplateWorksheet 和结果工作簿 ResultWorksheet 实现了工作表接口 IWorksheet，模板单元格 TemplateCellElement 和结果单元格 ResultCellElement 都实现了单元格接口 ICellElement。工作簿和工作表、工作表和单元格之间都是一对多的组成关系。

3.4 模板描述文件

对于报表引擎来说，最重要的就是模板的设计，它决定了报表的数据来源、数据处理过程和数据的展现方式。

报表模板也叫工作簿，它描述了一个完整的报表对象。一个报表模板的构成元至少包括数据来源、工作表、参数、样式列表。本文中的报表模板使用 xml 文件描



基于单元格扩展的报表引擎的研究与实现



述。一个工作簿的基本组成如下所示：

```
<Workbook>—————, 工作簿
  <DataSet>—————, 数据源集合
  ...
  </DataSet>
  <WorkSheetList>—————, 工作表列表
  ...
  </WorkSheetList>
  <Parameters>—————, 参数
  ...
  </Parameters>
  <StyleList>—————, 样式列表
  ...
  </StyleList>
</Workbook>
```

图 3-4 工作簿的基本组成示意图

一个工作簿对应一个 xml 文件。当一个工作簿文件被报表引擎解析之后，在服务器中生成一个对应的 TemplateWorkbook 对象。

所以，设计了一个工作簿就是创建了一个 TemplateWorkbook 对象，TemplateWorkbook 相当于一个容器，里面可以放任意个工作表，也就是 Worksheet 象，就像在 Excel 的一个工作簿中新建了多个 sheet 一样。而每个 Worksheet 是由多单元格 CellElement 组成，因此 CellElement 是一个模板的最小元素。如下图所示：

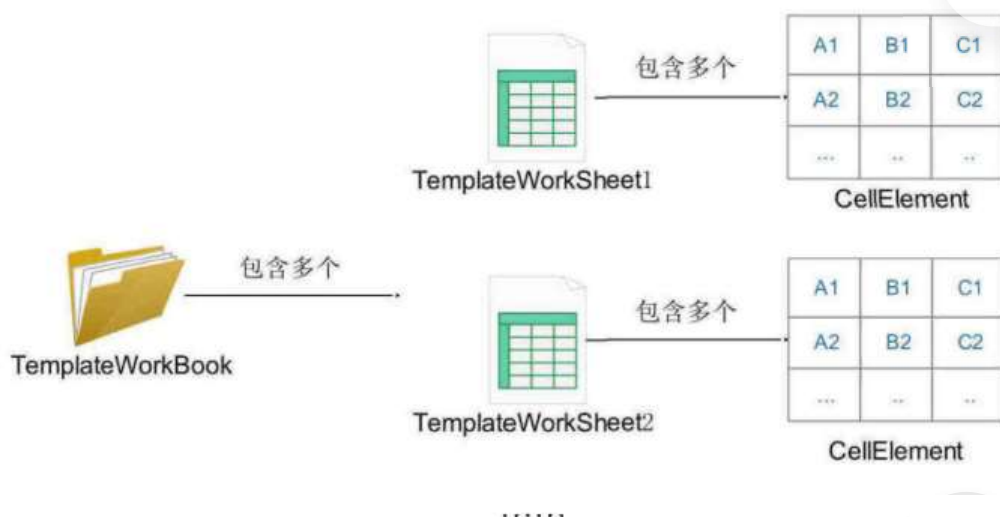


图 3-5 工作簿-工作表-单元格关系图

3.4.1 数据源

数据源定义了模板中的数据的来源，它的构成元素至少应包括使用的数据库连信息、数据库查询语句、参数等。一个模板中的数据可以来源于多个数据源，所以用 Set 来存储所有的数据来源。数据源的构成如下图所示：



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

第3章 数据源与数据表

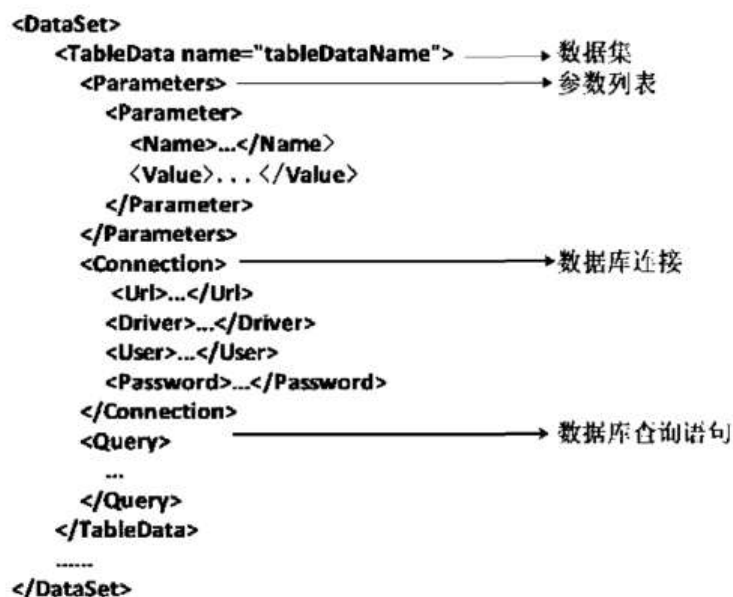


图 3-6 数据源的基本组成示意图

3.4.2 工作表

一个工作簿可以包含多个工作表。工作表定义了数据的展现形式。和 Excel 相关工作表的基本组成元素是单元格。此外，工作表还定义了页面属性信息，例如背景、页眉、页脚、是否居中展示等。工作表的构成如下图所示：

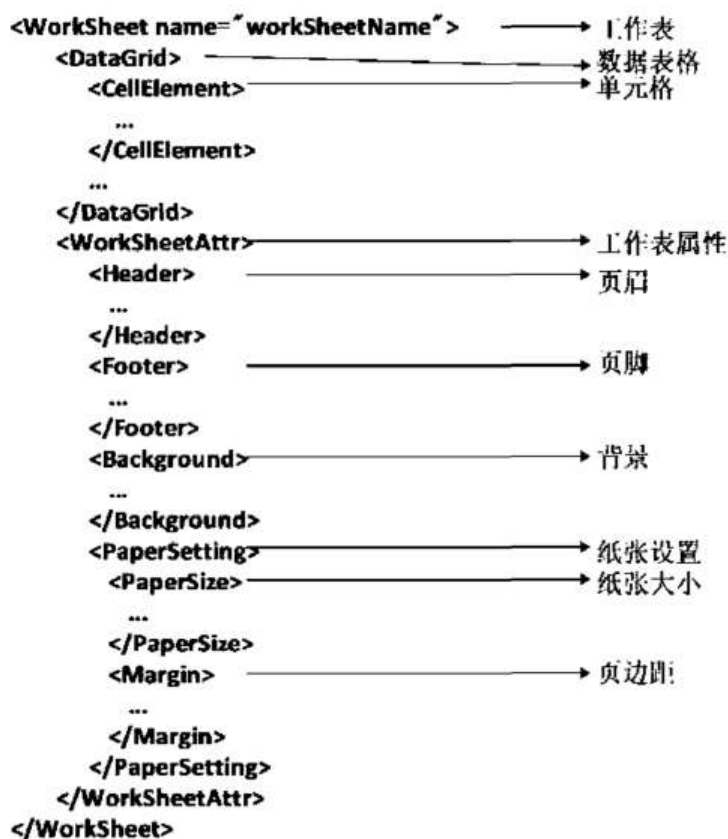


图 3-7 工作表的基本组成示意图





对于工作表来说，最基本的组成元素就是单元格。一个工作表中可以定义多个单元格。单元格的数据值来源有 3 种：静态数据（在定义工作表时已经确定单元格的数据值）、公式数据（需要进行计算后得到）、数据表数据（需要查询数据库得到具体值）。单元格元素至少要包含单元格行列位置、合并行列值、数据值、扩展属性等。单元格元素的构成如下图所示：

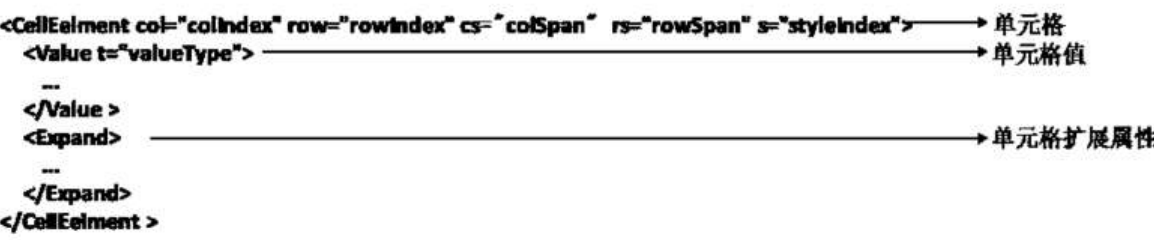


图 3-8 单元格的基本组成示意图

3.4.3 参数列表

一个工作簿可以定义参数、使用参数，这在报表展示中是很有必要的。很多时候数据的展示与参数值息息相关，例如报表的查询展示功能。有时，参数还要与数据源中的某个列的值绑定，例如下拉菜单中的数据。参数元素至少需要包含参数名、参数来源、默认值等属性。参数元素的构成如下图所示：

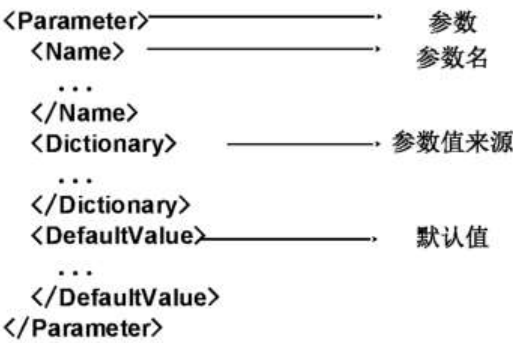


图 3-9 参数的基本组成示意图

3.4.4 样式列表

模板中每个单元格都可以有各自的样式。但是，对于一个模板来说，出于整体设计风格的考虑，大部分单元格的样式都是相同的。所以这里使用样式列表来管理工作簿中使用的所有样式，单元格只要声明使用的样式引用即可。事实证明，这是有效的。样式元素至少要包含字体、背景、边框等元素。样式列表元素的构成如下



基于单元格扩展的报表引擎的研究与实现





图 3-10 样式列表的基本组成示意图

3.5 结果工作簿

报表引擎对模板工作簿文档进行解析，生成工作簿 `TemplateWorkBook` 对象。然而，此刻的 `TemplateWorkBook` 对象是抽象的，单元格中的数据还没有计算、扩展和填充。当报表引擎对每个单元格执行查询、扩展、填充之后，会生成相应的结果工作对象。对应于 `TemplateWorkBook` 对象，结果工作簿对象 `ResultWorkBook` 的构成如下：

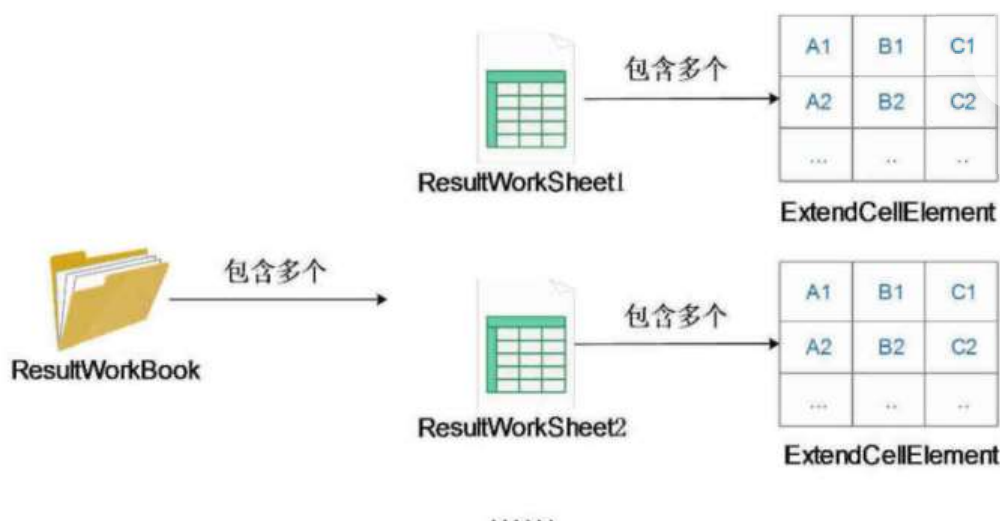


图 3-11 结果工作簿的构成示意图

3.6 报表引擎优化策略

3.6.1 WorkbookEntry

对于报表引擎服务器上的同一工作簿，可能先后会有多个用户访问。此时为了服务器的负载考虑，不必每次都重新去读取工作簿文档，所以可以多次请求共享第一次读取的结果。本文使用 `WorkbookEntry` 对象保存读取后解析的工作簿文档对象，使



基于单元格扩展的报表引擎的研究与实现

第三章 系统整体架构

WorkBookEntry 的工作原理如下：

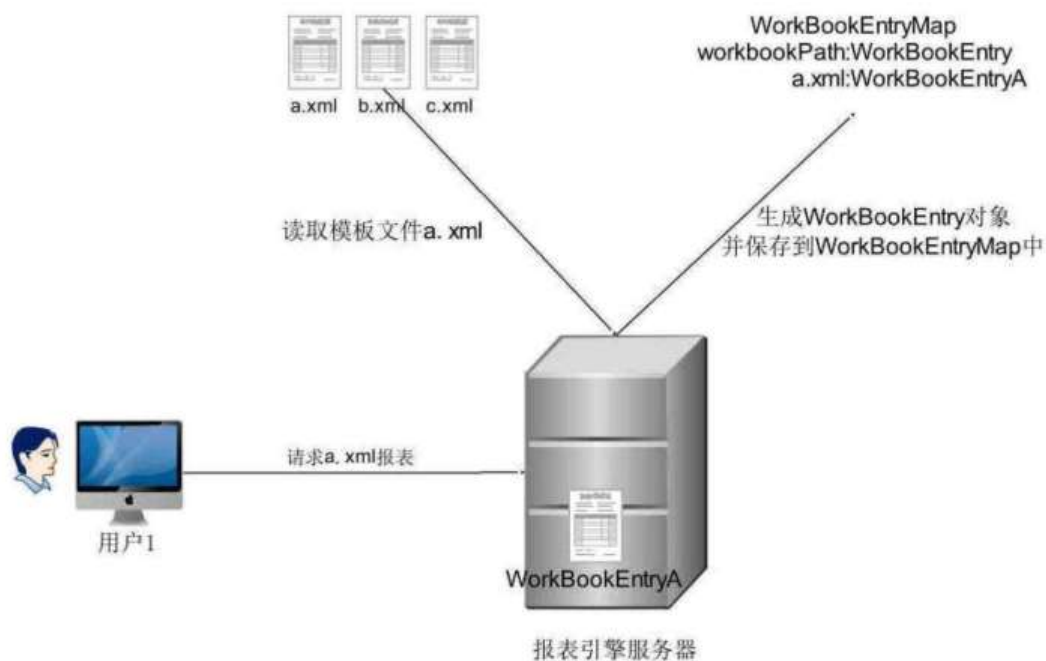


图 3-12 WorkBookEntry 的工作原理示意图 1

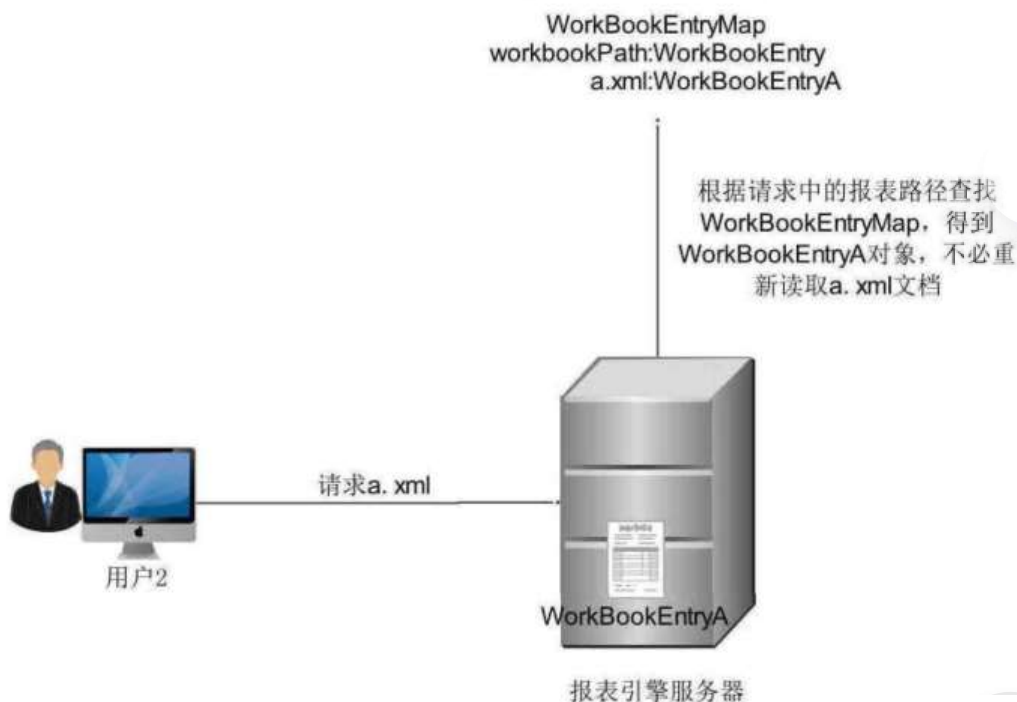


图 3-13 WorkBookEntry 的工作原理示意图 2



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

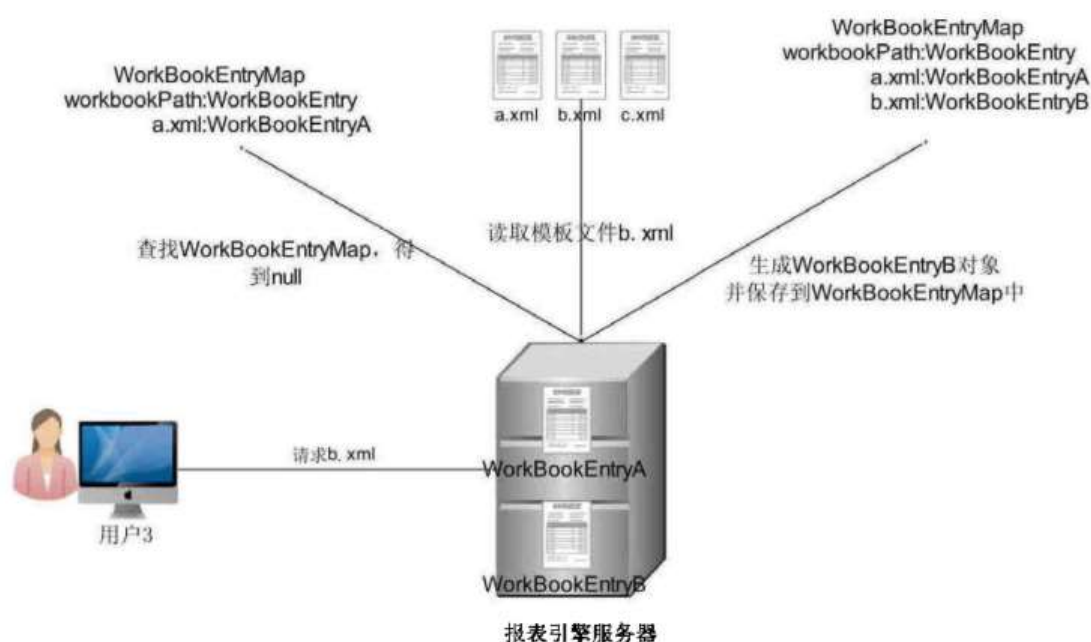
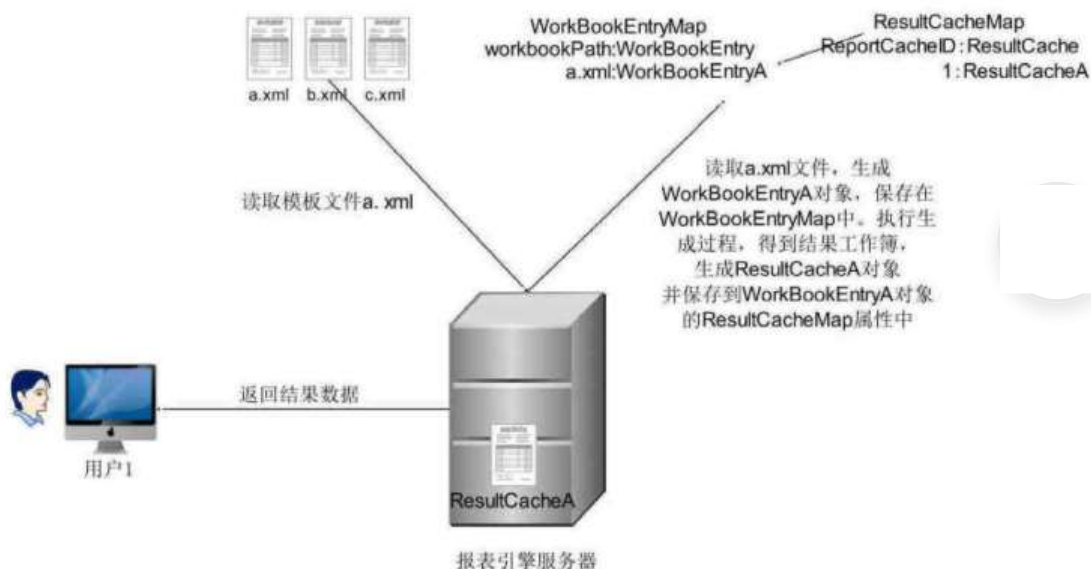


图 3-14 WorkbookEntry 的工作原理示意图 3

3.6.2 ResultCache

对于报表引擎服务器来说，最繁重的工作就是由模板工作簿对象 TemplateWorkbook 生成结果工作簿对象 ResultWorkbook 的执行过程。所以，与 ReportEntry 类似，如果某个工作簿已经执行了生成过程得到了结果工作簿对象 ResultWorkbook，那么对于其他对该工作簿的请求，应该共享第一次生成的结果。然，对于某些有时效性要求的报表，还需要检查该工作簿的生成时间是否过期。如未过期，可以继续使用生成的结果，否则需要重新生成结果工作簿。

本文使用 ResultCache 对象保存生成的结果工作簿对象，使用 ResultCacheMap 存<ResultCacheID:ResultWorkbook 对象>键值对。ResultCache 的工作原理如下所示





基于单元格扩展的报表引擎的研究与实现

第三章 系统整体架构

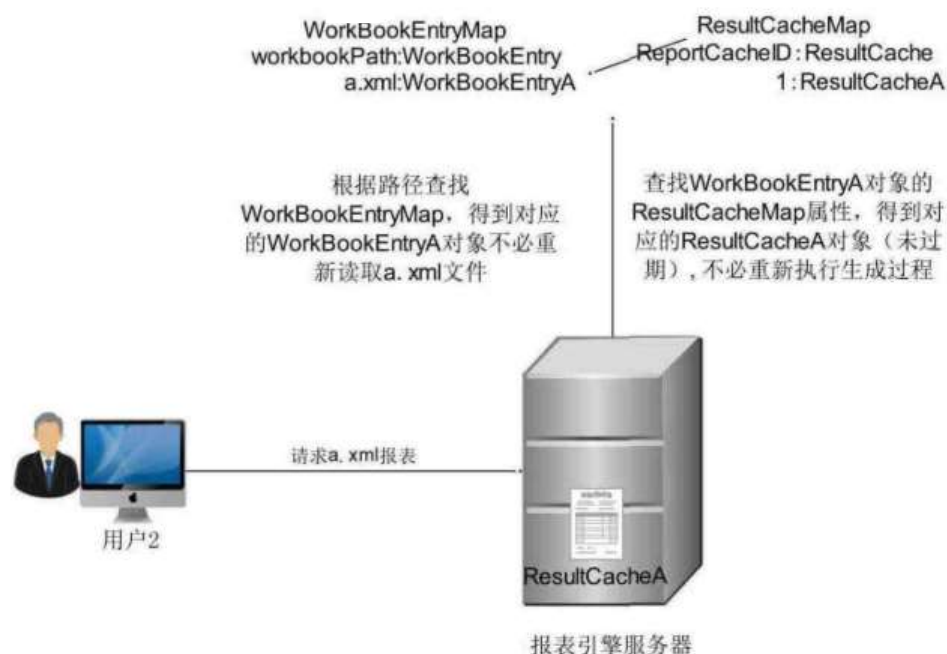


图 3-16 ResultCache 的工作原理示意图 2

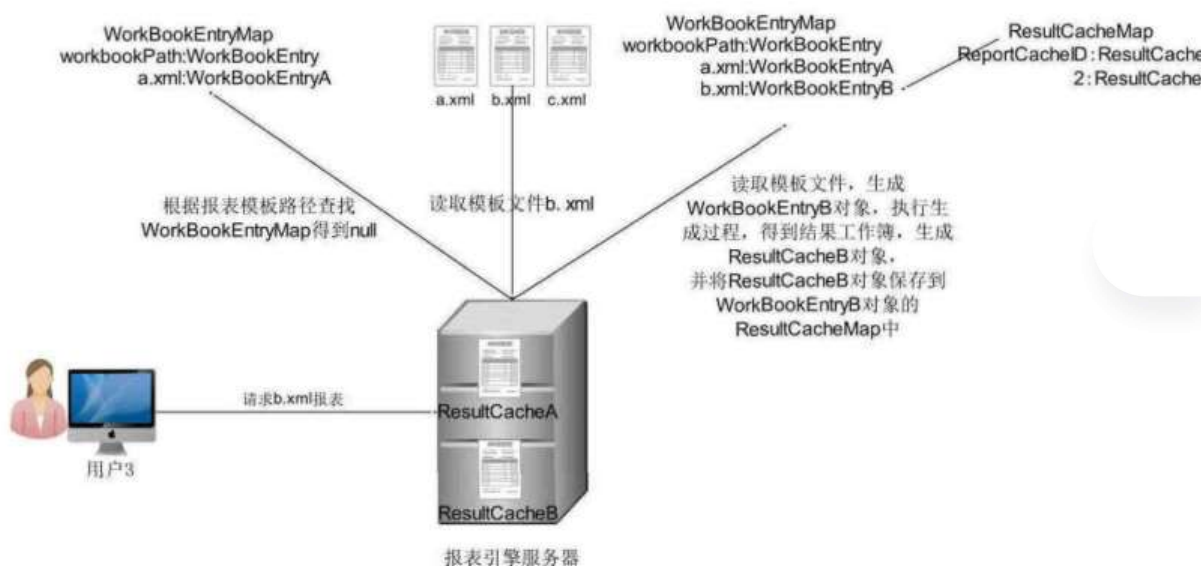


图 3-17 ResultCache 的工作原理示意图 3

3.6.3 SessionInf

对于访问报表的用户, 报表引擎不会一次性将所有页面信息都输出给客户端, 是只返回给客户端请求的页码对应的数据。所以, 报表引擎服务器需要记端访问信息, 以便客户端的下次请求的处理。

和 Servlet 的 Session 对象类似, 本文使用 SessionInf 对象保存客户端的信息。SessionInf 的具体实现如下:







图 3-18 SessionInfor 的属性示意图





第四章 单元格扩展原理

本章主要讲述怎样通过单元格的抽象 XML 定义来实现数据填充，最终得到一间结果矩阵的过程。

单元格的扩展是指单元格由一变多的现象，这种现象大多出现在数据库单元格中。在单元格的定义中，一个单元格的值是抽象的。尤其是对于数据库单元格来说单元格的值往往是对应于数据库的一列，事先并不知道查询数据库之后会得到多少值。然而一个单元格的往往是位于一行一列的一个单位，如果执行查询之后的结果大于 1，则需要将该单元格扩展至一行上的多个单元格或者一个列上的多个单元格（取决于单元格的扩展方向）。

而一个单元格进行了扩展，依赖于该单元格的子格也需要进行扩展，才能实现数据的一一对应。单元格进行扩展之后，扩展出来的每个单元格以及依赖于此单元格子格之间的继承关系也需要进行刷新。

单元格的扩展过程分为以下 5 步：

1. 计算单元格继承关系；
2. 初始化表格矩阵；
3. 填充表格矩阵；
4. 扩展子格、更新继承关系；
5. 计算最终结果矩阵；

本章以一张年级成绩单报表为例，详细描述了单元格扩展过程中每一步的具体表现。

4.1 报表设计

本章以一张年级成绩单报表为例，该报表使用了一张数据库表 score，报表设计有横向扩展单元格、纵向扩展单元格，是一张典型的交叉报表。同时，报表中应用了公式单元格。

score 有四列，分别为 class、studentName、subject、score。具体数据概览如下：



班级	姓名	科目	分数
一班	王亚楠	语文	122
二班	王英博	数学	127
一班	王亚楠	数学	127
二班	王英博	语文	116
一班	王亚楠	物理	84
二班	王英博	物理	86
二班	王英博	生物	88

图 4-1 score 表数据预览

报表的抽象布局如下表所示，具体的单元格定义在上一章已经进行了详述：

表 4-1 年级成绩表

	A	B	C	D
1	年级成绩表			
2				
3	科目/学生/班级		科目	总分
4	班级	学生	分数	SUM(C4)
5	平均分		AVERAGE(C4)	AVERAGE(D4)

各个单元格意义如下：

表 4-2 年级成绩表单元格定义表

A1	静态单元格，标题，合并了 2 行 4 列
A3	静态单元格，表头标题
C3	来自数据库中 score 表的 subject 列，横向扩展，分组显示
D3	静态单元格、表头标题
A4	来自数据库中 score 表的 class 列，纵向扩展，分组显示
B4	来自数据库中 score 表的 studentName 列，纵向扩展，分组显示
C4	来自数据库中 score 表的 score 列，无扩展，汇总显示
D4	公式单元格，对每个学生的分数进行汇总
A5	静态单元格
C5	公式单元格，对每个科目求平均分
D5	公式单元格，对所有学生的总成绩求平均分

4.2 计算单元格继承关系

单元格扩展的第一步就是要计算单元格之间的继承关系。因为父格的扩展对子有影响，子格的值也依赖于父格的值。所以，计算单元格之间的继承关系对初始化



格矩阵、填充表格矩阵、扩展子格、更新继承关系有重要的作用。

4.2.1 单元格的继承关系的过滤作用

我们定义一个单元格有两个父格，一个为行父格，一个为列父格。单元格的行格主要用于对单元格的行数据进行过滤，单元格的列父格主要用于对单元格的列数进行过滤。

一个单元格如果有行父格，那么该行父格必须是具有纵向扩展属性的单元格。就是说，只有具有纵向扩展属性的单元格才能作为行父格，才能对其子格有过滤作用。

例如，在上面的年级成绩表中，A4 单元格为纵向扩展的数据库单元格，B4 单元格的默认行父格为 A4。因此，当 A4 单元格执行数据库查询后，得到多个班级时，单元格的学生也要对应 A4 单元格的值进行计算、填充和展示。也就是说，属于一班的学生不能填充到二班所在的行后面。

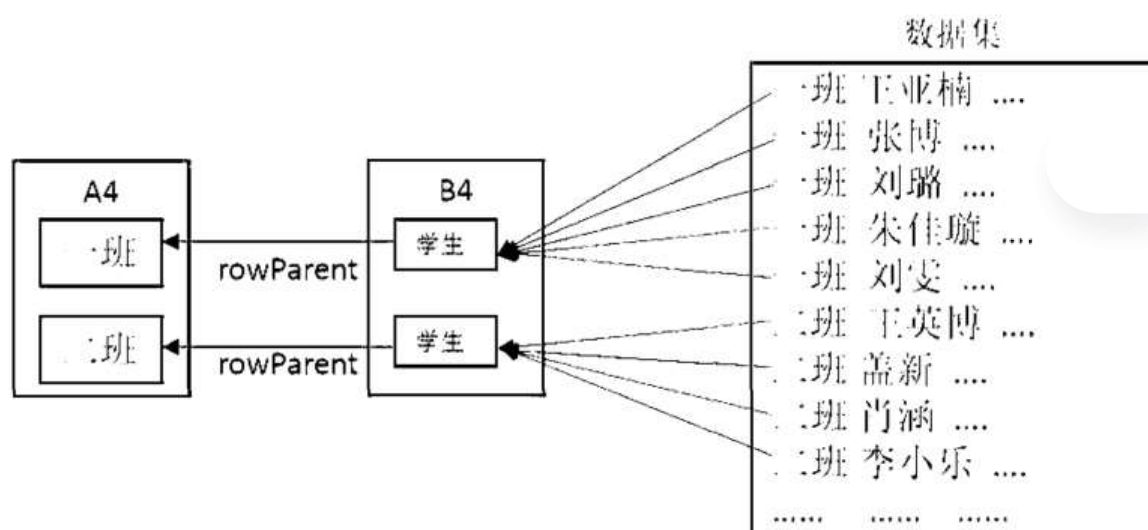


图 4-2 班级对学生的过滤作用示意图

而对于 D4 单元格，它的默认行父格是 C4。但是由于 C4 单元格并不具有纵向扩展属性，所以只能继续尝试 C4 单元格的行父格，也就是 B4，而 B4 单元格具有纵向扩展属性。所以，D4 的行父格是 B4 而不是 C4。也就是说，D4 是对具体的学生，而不是针对某个科目。这就是行父格必须是具有纵向扩展属性的单元格的原因，也是行父格对其子格的行数据进行过滤的体现。

同样的，一个单元格如果有列父格，那么该列父格必须是具有横向扩展属性的单元格。也就是说，只有具有横向扩展属性的单元格才能作为列父格，才能对其子格

基于单元格扩展的报表引擎的研究与实现



例如，在上面的年级成绩表中，C3 单元格为横向扩展的数据库单元格，C4 单元格的默认列父格为 C3。因此，当 C3 单元格执行数据库查询后，得到多个科目时，C4 单元格中的分数也要对应 C3 单元格的值进行计算、填充和展示。也就是说，语文的分数不能填充到数学所在的列下面。

而对于 C5 单元格，它的默认列父格是 C4，但是由于 C4 单元格并不具有横向扩展属性，所以只能继续尝试 C4 单元格的列父格，也就是 C3，而 C3 单元格是具有横向扩展属性的单元格。所以，C5 的行父格是 C3 而不是 C4。也就是说，C5 的求平均是对某个科目的。这就是列父格必须是具有横向扩展属性的单元格的原因，也是列父格对其子格的列数据进行过滤的体现。

而对于 D5 单元格，它的默认行父格和默认列父格都不具有纵向扩展属性或横向扩展属性，所以 D5 单元格的列父格和行父格经计算后为空。

4.2.2 计算单元格的继承关系

单元格的行父格计算步骤如下：

1. 获得当前已定义表格的行数 `rowCount`，新建一个 `rowParentArray` 数组用来存储每一行对应的行父格，该数组的长度为 `rowCount`，该数组中每一项的初值都为 `null`；
2. 遍历已定义的每一个单元格 `cellElement`，得到该单元格的扩展属性，如果单元格的具有默认行父格，转到 3，否则，转到 4；
3. 得到该单元格 `cellElement` 所在的列 `row` 和合并行数 `rowSpan`，查询 `rowParentArray` 数组中 `[row,row+rowSpan]` 上的元素，直到找到某个单元格 `rowParent` `rowParent!=null` 且 `rowParent!=cellElement`，将 `rowParent` 设置为 `cellElement` 的行父格，如果 `rowParent=null` 或 `rowParent!=cellElement`，则将 `cellElement` 的行父格设置为 `null`，转到 4；
4. 如果当前单元格 `cellElement` 具有纵向扩展属性，得到该单元格所在的行 `row` 和合并行 `rowSpan`，将 `rowParentArray` 序列的 `[row,row+rowSpan]` 位置上的元素设置为当前单元格 `cellElement`，转到 5；如果当前单元格 `cellElement` 不具有纵向扩展属性，转到 5；
5. 转到 2，直到遍历结束；

单元格的行父格计算的伪代码如下所示：



基于单元格扩展的报表引擎的研究与实现



	calculateLeftParent
Input:	抽象表格的行数 rowCount
Output:	无
1:	rowParentArray = new CellElement[rowCount];
2:	For(表格中的每个抽象单元格 cellElement)
3:	获取 cellElement 的扩展属性 cellExpandAttr;
4:	If(cellElement 有默认行父格)
5:	得到 cellElement 所在的行 row 和合并行 rowSpan, 遍历 rowParentArray 序列的 row 至 row+rowSpan 位置上的元素, 直到找到某个元素 rowParent != null 且 rowParent != cellElement;
6:	If(rowParent != null && rowParent != cellElement)
7:	设置 cellElement 的行父格为 rowParent;
8:	Else
9:	设置 cellElement 的行父格为 null;
10:	If(cellElement 有纵向扩展属性)
11:	得到 cellElement 所在的行 row 和合并行 rowSpan, 将 rowParentArray 序列的 row 至 row+rowSpan 位置上的元素设置为当前单元格 cellElement;
12:	End

单元格的列父格计算步骤同理。

根据上述的计算规则，年级成绩表中各单元格继承关系计算后的结果如下图所示：

A1	-	-	-
-	-	-	-
A3	-	C3	D3
A4	B4	C4	D4
A5	-	C5	D5





4.2.3 继承关系矩阵组成

上一节详细描述了整个表格中单元格的继承关系的计算过程，但是单元格的继承关系是存储在单元格的属性字段中的，查找很不方便，而且，只能由子节点查询到节点，不能由父节点得到所有依赖它的子节点。所以，我们使用一个继承关系矩阵（genealogy）来描述单元格的继承关系，以后单元格扩展的每一步都与这个继承关系矩阵密不可分。

genealogy 矩阵中的每一个元素都是 FamilyMember 对象，该对象有四个与继承关系有关的属性：

1. selfCellElement: 用来存储某个具体的单元格对象的引用；
2. rowParent: FamilyMember 类型，用来存储该单元格的行父格引用；
3. columnParent: FamilyMember 类型，用来存储该单元格的列父格引用；
4. childMembers: FamilyMember 类型的链表，用来存储该单元格的所有子格的引用；

4.2.4 计算继承关系矩阵

genealogy 矩阵的构建过程如下：

1. 初始化一个 FamilyMember 类型的 genealogy 二维矩阵，该矩阵的横向长度表格的列数，该矩阵的纵向长度为表格的行数；初始化一个 HashSet 类型的集合 set，用于防止计算过程中出现的死循环；
2. 遍历每一个单元格，执行 3；
3. 如果 genealogy 矩阵中对应该单元格的行列位置不为空，返回该位置的 FamilyMember 对象；否则，转到 4；
4. 如果 set 中已经包含了该单元格，停止计算，说明继承关系出现了死循环，出异常；否则，转到 5；
5. 将该单元格放入 set 中，如果该单元格的属性中的行父格不为空，父的继承关系，执行 3，得到行父格对应的 FamilyMember 对象 rowParentMember；如果该单元格的属性中的列父格不为空，计算列父格的继承关系，执行 3，得到列父格对应的 FamilyMember 对象 columnParentMember，转到 6；
6. 新建一个 FamilyMember 对象 currentMember，将该对象的 selfCellElement





基于单元格扩展的报表引擎的研究与实现

第四章 单元格扩展原理

性设置为当前单元格、rowParent 属性设置为 rowParentMember、columnParent 属性设置为 columnParentMember，将 genealogy 矩阵中对应该单元格的行父对象设置为该对象，将 currentMember 对象添加到 rowParentMember、columnParentMember 的 childMembers 链表中，返回 currentMember 对象；上述过程的伪代码如下：

计算继承关系矩阵的伪代码

Input: 抽象表格的总行数 rowCount

抽象表格的总列数 columnCount

Output: 继承关系矩阵 genealogy

```
1: genealogy = new FamilyMember[rowCount][columnCount];
   set = new HashSet();
2: For(表格中的每个抽象单元格 cellElement)
3:     buildGenealogy(cellElement,set);
4: End
```

buildGenealogy

Input: 单元格对象 cellElement

HashSet 对象 set

Output: cellElement 对应的 FamilyMember 对象

```
1: 获取 cellElement 的所在的行 row 和列 column;
2: If ( genealogy[row][column] != null )
3:     返回 genealogy[row][column];
4: If(set 已经包含了当前单元格 cellElement)
5:     抛出死循环异常;
6: 将 cellElement 放入 set 中;
7: rowParentMember = null;
   columnParentMember = null;
8: If(cellElement 的行父格不为空)
9:     调用 buildGenealogy(rowCellElementParent)得到当前单元格的行父格
   对应的 FamilyMember 对象并赋给 rowParentMember;
10: If(cellElement 的列父格不为空)
11:     调用 buildGenealogy(columnCellElementParent)得到当前单元格的列父
```





```

-- -- -- -- -- 格对应的 FamilyMember 对象并赋给 columnParentMember;
12: -- -- -- 创建一个当前单元格对应的 FamilyMember 对象 currentMember, 该
      currentMember 的 selfCellElement 属性为 cellElement、rowParent 属性为
      rowParentMember、columnParent 属性为 columnParentMember;
13: -- -- -- 将 genealogy[row][column]的值设置为 currentMember;
14: -- -- -- 从 set 中移除当前单元格对象 cellElement;
15: -- -- -- 将 currentMember 加入到父格的 rowParentMember、columnParentMember
      对象的 childMembers 序列中;
16: -- -- -- 返回 currentMember

```

经过计算，地区销售概况表中各单元格的继承关系如下所示：

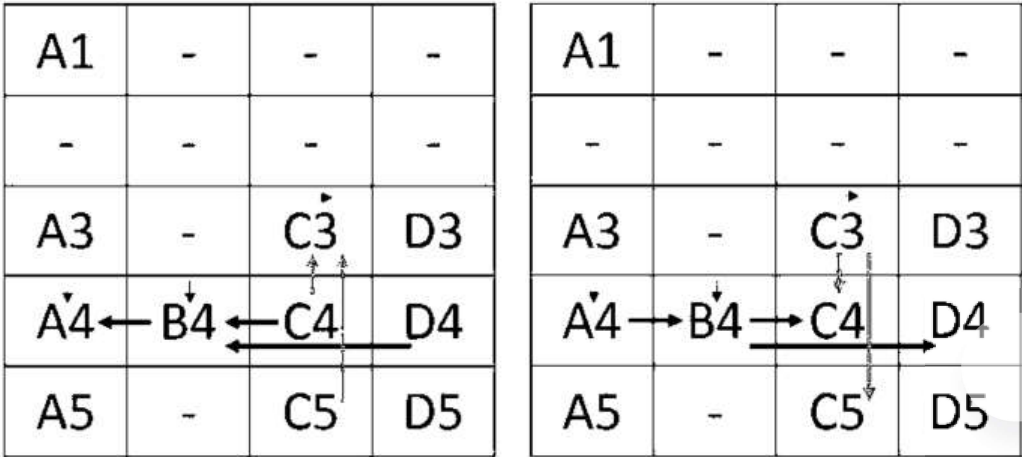


图 4-4 年级成绩表单元格父关系（左）子关系（右）示意图

4.3 初始化表格矩阵

在上一步中，我们得到了整个表格中单元格的继承关系矩阵 genealogy。该矩阵是用来描述当前表格中各单元格的继承依赖关系的，我们还需要一个容器用来装载表格中每个单元格计算、扩展的值。我们称之为表格矩阵。

4.3.1 表格矩阵组成

首先，对于一个单元格来说，它的值类型可以分为两类：一类是非扩展数据，就是在定义单元格时已经确定了单元格的值，静态数据单元格就属于这种情况；一是可扩展数据，也就是在定义单元格时并不知道单元格的值，需要经过进一步的计或查询数据库才能得到具体值，数据库单元格就属于这种情况。我们使用两种类型描述这两种情况。对应非扩展数据单元格，我们用 NormalBoxCE 类型来描述，对于





基于单元格扩展的报表引擎的研究与实现

第四章 单元格扩展原理

和 ExtendedBoxCE 类型的数据对象是组成表格矩阵的最基本也是最重要的元素，因为只有它们维护着单元格的具体值。但是它们也仅仅保存单元格的具体值，不保存单元格的继承关系。

如果一个单元格的值是可扩展数据，那么我们事先并不知道它会得到多少具体值，也就是说我们无法确定这个单元格会扩展到什么大小。因此，我们需要一个对象来管理这些扩展出来的数据对象，这就是 BE 对象。BE 对象中有一个 List 类型的 extendedBoxCEList 属性，用来存放该单元格扩展出来的所有数据。BE 除了保存扩展出来的数据对象，还保存这些扩展数据对象的继承关系。

BE 对象用以下属性保存继承关系：

1. rowPE:保存当前单元格的行父格引用；
2. columnPE:保存当前单元格的列父格引用；
3. extendedBoxCEList:保存当前单元格的扩展格序列；
4. chilBEs:保存当前单元格的子格序列；

然而，仅仅使用 BE 对象来代表一个单元格也是有问题的。对于某些有多层继承关系的单元格来说，如果它通过计算得到了多于一个的值，假设是 N 个值，那么它的格也至少需要扩展到 N 个，每个子格的父格分别设置为这 N 个值，以达到数据的一一对应。

那么此时，仅仅使用 BE 对象来代表一个单元格已经无法满足这种情况，因为 BE 对象只能保存一个行父格或列父格引用，无法保存 $N(N>0)$ 个行父格或列父格以到一一对应的效果。因此，我们定义了一个更高层次的类型 BEB 来管理这些扩展出的 BE 类型。BEB 类型有一个 List 类型的 beList 属性，用来存储由于父格扩展而产生的 BE 对象。

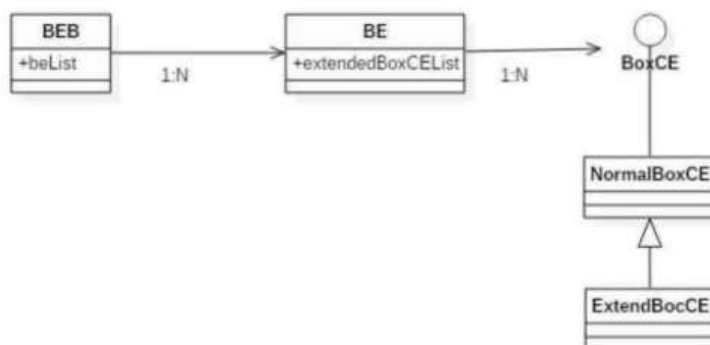


图 4-5 BEB- BE-BoxCE 关系图





综上所述，一个抽象的单元格对应一个 BEB 对象，这个对象只是用来管理由于格扩展而导致的子格扩展。每个 BEB 有一个 beList 属性用于存储由于父格扩展而导致的子格扩展出来的对象，也就是 BE 对象。最初，每个 BEB 的 beList 属性都有一个对象，该 BE 对象的 extendedBoxCEList 属性为空。在单元格的计算过程中，如果该单元格为非扩展数据单元格，那么会初始化一个 NormalBoxCE 对象放入该单元格对应的 BEB 对象的 beList 属性中的 BE 对象中的 extendedBoxCEList 属性当中，如果该单元格是可扩展数据单元格，那么计算之后，对于得到的每一个具体值，都会初始化一个 ExtendedBoxCE 对象，并把这些对象放入该单元格对应的 BEB 对象的 beList 属性中的 BE 对象中的 extendedBoxCEList 属性当中。如果一个 BE 对象通过计算得到多个具体值，那么依赖它的子格的 BE 对象也需要由一个扩展到多个，并把扩展出来的对象放入子格对应的 BEB 的 beList 属性当中。

4.3.2 初始化表格矩阵

上一节详细描述了表格矩阵对象的组成。有了继承关系矩阵 genealogy 和表格对象的概念，我们就可以构建一个表格矩阵用来存储表格的计算和扩展得到的数据。表格矩阵的构建过程如下：

1. 初始化一个 BEB 类型的 be_beb_2D 维矩阵，该矩阵的横向长度为表格的列数，该矩阵的纵向长度为表格的行数；
2. 遍历 genealogy 矩阵，执行 3；
3. 得到当前 FamilyMember 对象的 selfCellElement 属性，即当前单元格 currentCellElement，如果 be_beb_2D 矩阵中对应该单元格的行列位置不为空，返回该位置的 BEB 对象；否则，转到 4；
4. 如果当前 FamilyMember 对象的 rowParent 属性不为空，计算行父格的 BEB，执行 3，得到行父格对应的 BEB 对象的 beList 属性中存储的 BE 对象 rowBE；如果当前 FamilyMember 对象的 columnParent 属性不为空，计算列父格的 BEB，执行 3，得到列父格对应的 BEB 对象的 beList 中存储的 BE 对象 columnBE；转到 5；
5. 初始化一个 BE 对象 be，将该对象的行父格 rowPE 属性设置为 rowBE，列父格 columnPE 属性设置为 columnBE；初始化一个 BEB 对象 beb，将 be 添加到该对象的 beList 当中，并将该对象的 cellElement 属性设置为





为 beb；转到 6；

6. 如果当前 FamilyMember 对象的 childMebers 属性不为空，初始化当前 be 的 childBEs 为 childMebers 长度的 BE 类型的数组；如果 rowBE 不为空，将 b 加到 rowBE 的 childBEs 中；如果 columnBE 不为空，将 be 添加到 columnBE 的 childBEs 中；返回 beb 对象；

经过上述计算，年级成绩表的表格矩阵初始状态如下：

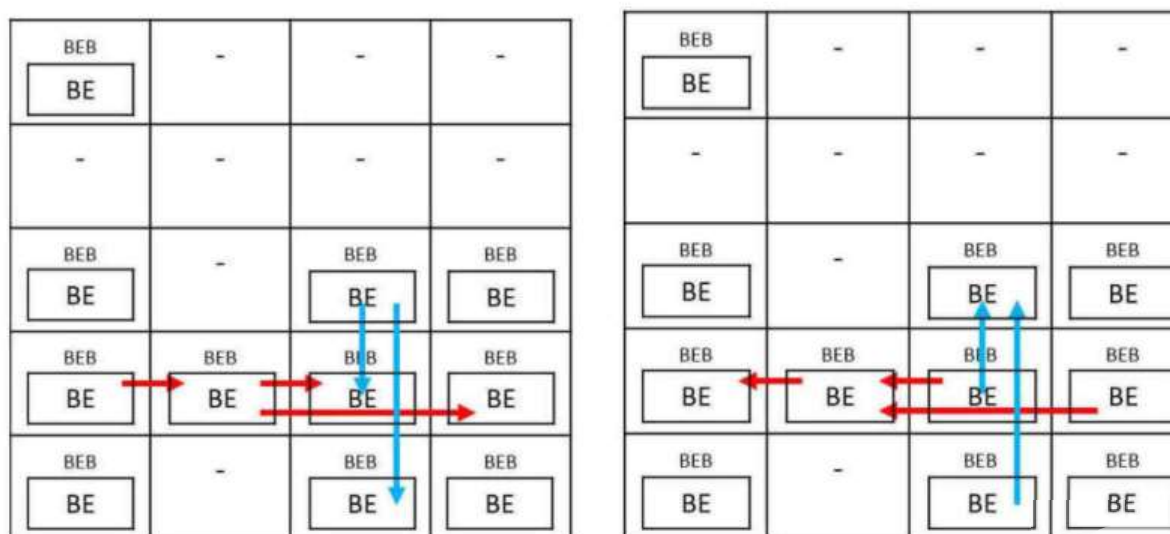


图 4-6 childBEs（左）parent（右）示意图

4.4 填充表格矩阵

上一节详细描述了如何根据 genealogy 矩阵创建表格矩阵 be_beb_2D。该矩阵用装载表格中每个单元格计算、扩展的值。be_beb_2D 矩阵初始化完成之后，就要执行矩阵的填充了。在表格的填充过程中，会依次计算各个单元格的值，然后根据计算结果执行单元格的扩展。

表格矩阵的填充顺序是根据单元格在表格中的位置，从左到右，从上到下依次行，如果当计算的单元格有父格，那么先计算父格的值，再计算子格的值。为了标记某个单元格是否已经被计算过了，这里使用一个二维 bool 矩阵 isCalStateMatrix 来标记这个单元格是否已经被填充过了。初始时，isCalStateMatrix 矩阵的每个位置都是 false，当某个单元格被填充过了，则将对应该位置上的值标记为 true。

为了防止在计算过程中出现单元格的值循环引用而引起的死循环，这里使用一个 Set 类型的 isCalculatingSet 来预防死循环。当计算某个单元格的值时，将该单元格放入 isCalculatingSet 集合中，如果该单元格有行父格或列父格，那么先计算父格的值，同样也将父格放入 isCalculatingSet 集合中。在计算完父格的值之后，再计算子格的值。





基于单元格扩展的报表引擎的研究与实现

每次计算完一个单元格的值，都将该单元格从 isCalculatingSet 集合中移除。每次开始计算一个单元格的值时，先判断 isCalculatingSet 集合中是否已经存在该单元格，如包含，则出现了循环引用，导致了死循环，此时停止计算，抛出死循环异常。

— — — — 表格的填充过程如下所示：

1. 初始化 isCalStateMatrix 矩阵用来标记单元格是否被计算过，初始化 isCalculatingSet 集合用来防止在计算过程中出现死循环；
2. 遍历每一个单元格，执行 3；
3. 如果 isCalStateMatrix 矩阵中该单元格 cellElement 所在的位置上标记为 true 则该单元格已经被计算过，直接返回；否则，检查 isCalculatingSet 集合中是否已经存在该单元格，如果存在，结束表格的填充，抛出死循环异常；如不存在，将该单元格放入 isCalculatingSet 集合中，转到 4；
4. 如果该单元格的行父格 rowParent 不为空，先计算行父格的值，执行 3；如该单元格的列父格 columnParent 不为空，先计算列父格的值，执行 3；
5. 从 be_beb_2D 矩阵中得到该单元格对应的 BEB 对象 beb，通过 beb 的 beList 属性得到 beList 链表，执行 6；
6. 对于 beList 链表中的每一个 BE 对象，执行 7；
7. 对于当前需要计算的 be 对象，得到该 be 所在的单元格的值类型，如果是扩展数据单元格，则初始化一个 NormalBoxCE 对象，存储该单元格的值，然后将 boxCE 添加到 be 的 extendedBoxCEList 列表中；如果是可扩展数据单元格，即数据库单元格或公式单元格，查询数据库或计算公式的值，得到一个或多个数据，对于每个数据，都初始化一个 ExtendBoxCE 对象，然后将这些对象都添加到 be 的 extendedBoxCEList 列表中；转到 8；
8. 扩展子格、更新继承关系（下一节详细讲述）；
9. 从 isCalculatingSet 集合中移除该单元格，将 isCalStateMatrix 矩阵中该单元格所在的位置标记为 true，返回；

上述填充过程的伪代码如下：

calCellElements	
Input:	单元格集合 elementCase
Output:	填充数据的表格矩阵 be_beb_2D
1: isCalStateMatrix = new boolean[rowCount][columnCount];	

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

第四章 单元格扩展原理

```
isCalculatingSet = new HashSet();
2: For(elementCase 集合中的每个 cellElement)
3:     calCellElementAndSetState (cellElement);
4: End
```

calCellElementAndSetState

Input: 标记矩阵 isCalStateMatrix
用于防止产生计算死循环的 isCalculatingSet
抽象单元格对象 cellElement
继承矩阵 genealogy
表格矩阵 be_beb_2D

Output: 抽象单元格对应的 BEB 对象

```
1: 获取 familyMember 保存的 cellElement 对象, 获取当前单元格所在的行 row
   和列 column;
2: If(isCalStateMatrix[row][column] =true )
3:     则该单元格已经被计算过了, 直接返回;
4: If(isCalculatingSet 已经包含该单元格)
5:     计算出现了死循环引用, 抛出死循环异常;
6: 将当前单元格加入 isCalculatingSet 中;
7: 获取该单元格对应的 FamilyMember 对象 familyMember;
8: If(familyMember 的行父格不为空)
9:     调用
       calCellElementAndSetState (familyMember.rowParent.seftCellElement)
       计算当前单元格的行父格的值;
10: If(familyMember 的列父格不为空)
11:     调用
       calCellElementAndSetState
       (familyMember.columnParent.seftCellElement)计算当前单元格
       格的值;
12: 获取该单元格对应的 BEB 对象 be;
13: 获取当前单元格 be 对象包含的 BE 对象列表 beList;
14: 调用 calBoxElements(beList);
15: 扩展子格, 更新继承关系;
```





```
17:  将 isCalStateMatrix[row][column] 设置为 true;

calBoxElements

Input:  每个单元格 BEB 对象包含的 beList
Output: 无

1:  For(beList 集合中的每个 be 对象)
2:      计算每个 be 对象的值, 并保存在 be 对象的 extendedBoxCEList 中;
3:  End
```

根据上述过程，年级成绩表中的单元格 A1、A3 的填充结果如下所示：

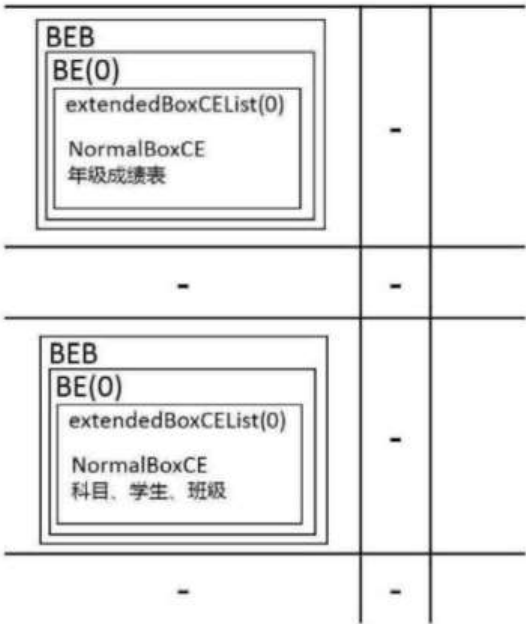


图 4-7 A1A3 填充结果示意图

单元格 C3 的填充结果如下所示：

BEB					
BE(0)					
extendedBoxCEList(0)	extendedBoxCEList(1)	extendedBoxCEList(2)	extendedBoxCEList(3)	extendedBoxCEList(4)	extendedBoxCEList(5)
ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE
语文	数学	英语	物理	化学	生物

图 4-8 C3 填充结果示意图

4.5 扩展子格、更新继承关系

上一节中详细描述了单元格的填充过程。单元格的填充过程和扩展子格、更新继承关系过程是密不可分的。每当填充了一个单元格，就要立刻进行子格的扩展、更新继承关系。这在上一节的填充过程中可以看到。





4.5.1 扩展子格

根据上一节，年级成绩表中的 C3 填充结果如下：

BEB					
BE(0)					
extendedBoxCELlist(0)	extendedBoxCELlist(1)	extendedBoxCELlist(2)	extendedBoxCELlist(3)	extendedBoxCELlist(4)	extendedBoxCELlist(5)
ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE	ExtendBoxCE
语文	数学	英语	物理	化学	生物

图 4-9 C3 填充结果示意图

所以科目单元格执行填充之后，由原本抽象的 1 列变成了 6 列，那么它的子格以及各科的平均分也必须由 1 列扩展到 6 列。同理，当班级单元格执行了填充之后，原本抽象的 1 行变成了 2 行，那么它的子格学生、分数、总分也必须由 1 行扩展到 2 行。

4.5.2 更新继承关系

如果一个 BE 对象 be1 执行了数据库查询，那么它的 extendedBoxCELlist 属性中填充具体的 n 个数据；如果一个 BE 对象 be2 为 be1 对象的子格，那么它没有具体的数据可以填充，只能将本身拷贝出 n-1 份，即扩展到 n 个 BE 对象；如果一个 BE 对象 be3 为 be2 对象的子格，那么它也会执行类似 be2 的扩展操作。

上述这两种情况在更新继承关系时是不同的，具体表现在，be1 更新继承关系时，be2 扩展出的 n 个 BE 对象的父格都要分别设置为 be1 的 extendedBoxCELlist 链表中各个有具体值的单元格，be1 的 childBEs 对象为 be2 扩展出的 n 个 BE 对象，所以 be1 和 be2 为一对多的关系，这种扩展方式叫直接更新；而 be2 更新继承关系时，由于 be2 和 be3 都扩展出了 n 个 BE 对象，所以，be3 扩展出的 n 个 BE 对象和 be2 扩展出的 n 个 BE 对象分别对应设置继承关系，所以 be2 和 be3 是一一对应的关系。此外，对于 be2 或 be3，它可能是交叉单元格（既有列父格，又有行父格），所以在设置一个方向上的继承关系之后，还要设置另一个方向上的继承关系。例如，如果 be2 既有列父格 be4 又有行父格 be4，那么，在设置完 be1、be2 之间的继承关系之后，也要讲 be2 扩展来的 n 个 BE 对象设置到 be4 的 childBEs 属性中，这种更新方式叫间接更新。

对应上面的两种情况，这里使用 DirectExtendedBEContainer 类来处理直接更新情况，使用 IndirectExtendedBEContainer 类处理间接更新的情况。

直接更新和间接更新对应于年级成绩表中的 A4、B4、C4 单元格的扩展更新过如下所示：

A4 的数据填充：





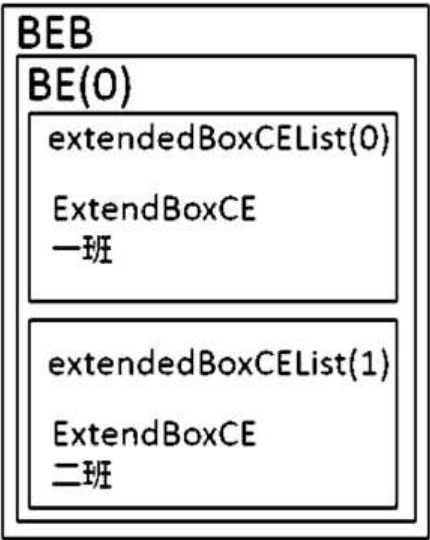


图 4-10 A4 填充结果示意图

A4、B4 的直接更新：

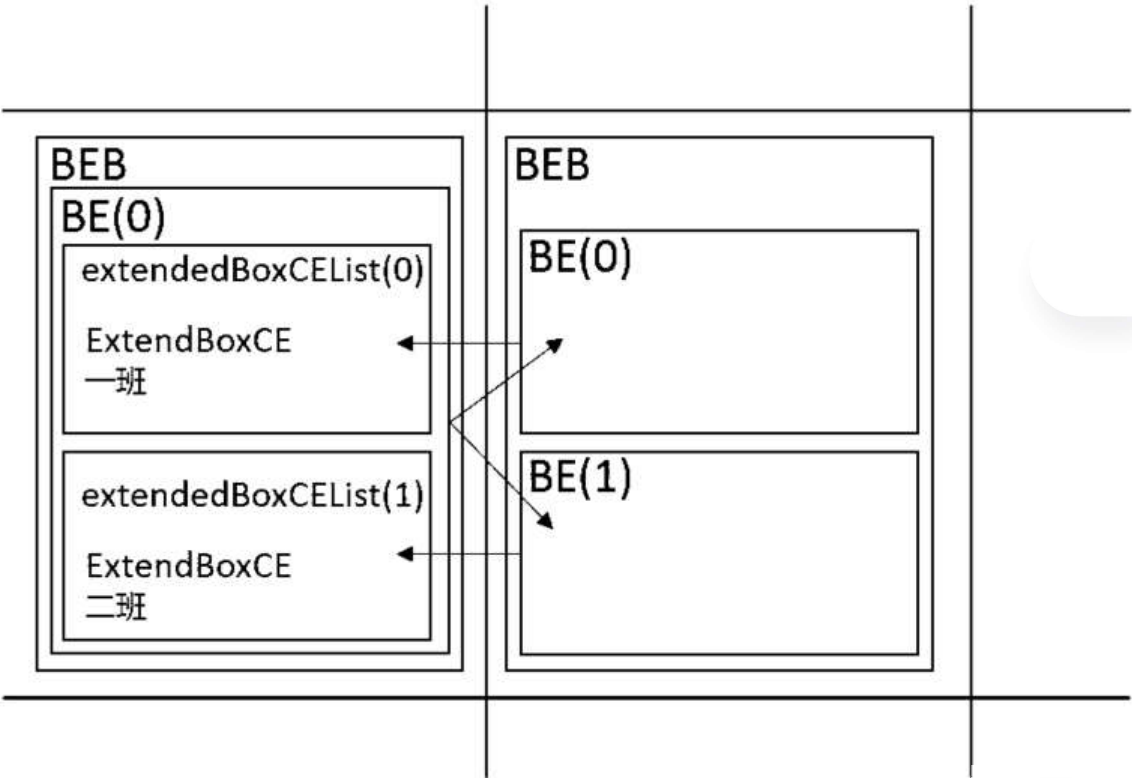


图 4-11 A4B4 的直接更新示意图

B4、C4 的间接更新



基于单元格扩展的报表引擎的研究与实现



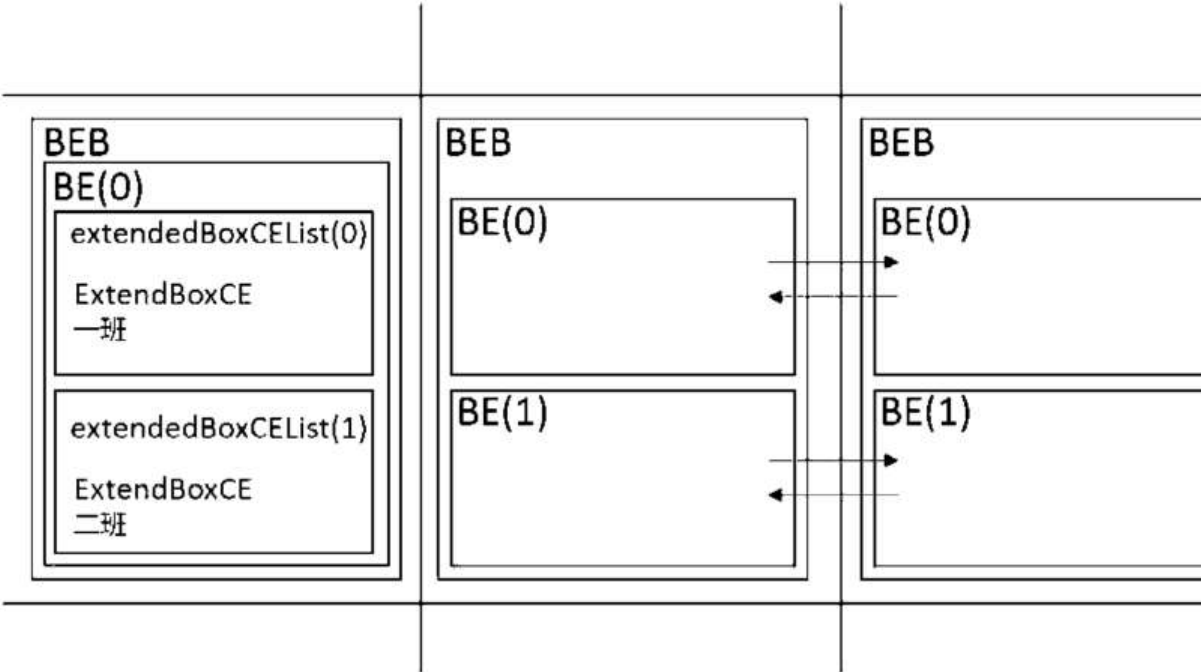


图 4-12 B4C4 的间接更新示意图

4.5.3 扩展、更新过程

扩展子格、更新继承关系的具体过程如下：

1. 遍历每一个单元格，执行 2；
2. 从 be_beb_2D 矩阵中得到该单元格对应的 BEB 对象 beb，通过 beb 的 beLi 属性得到 beList 链表，遍历 beList 链表中每一个 BE 对象，计算 BE 的值，充到 BE 对象 extendedBoxCEList 属性中，转到 3；
3. 初识化一个空的 cellElementExtendBEsMap，遍历 beList 链表中每一个 BE 对象 be，执行 4，完成遍历后，执行 9；
4. 对于每一个 be 对象，初始化一个空的 beExtendBEsMap，首先将<be，由 be.extendedBoxCEList 初始化的 DirectExtendedBEContainer 对象>键值对放入 beExtendBEsMap 中；转到 5；
5. 遍历 be 对象的 childBEs 属性，将每个 childBE 对象扩展到和 be.extendedBoxCEList.size 一样多，将<childBE，由 be.extendedBoxCEList 初始化的 IndirectExtendedBEContainer 对象>键值对放入 beExtendBEsMap 中，将<childBE 保存的 cellElement 对象，be.extendedBoxCEList.size 个 childBE 链表>键值对放入 cellElementExtendBEsMap 中，对于每个 childBE，递归执行 5，直到继承链结束。转到 6。





基于单元格扩展的报表引擎的研究与实现

6. 遍历 beExtendBEsMap 中的每个 value，即每个由扩展对象构成的 DirectExtendedBEContainer 对象或 IndirectExtendedBEContainer 对象，执行或 8；
7. 如果该 value 是 DirectExtendedBEContainer 对象，说明该 value 对应的 key 直接更新的单元格，该 value 中存储的 extendCellList 都是具体的数据；那么，只需要遍历 key（也就是 BE 对象）的 childBEs 对象，从 beExtendBEsMap 中取出每个 childBE 子节点对应的 IndirectExtendedBEContainer 对象中存储的拷贝单元格链表 childExtendBEList，将 extendCellList 和 childExtendBEList 一一对应即可，后，将这些拷贝单元格链表都放入 be 对象的 childBEs 属性中；
8. 如果该 value 是 IndirectExtendedBEContainer 对象，说明该 value 对应的 key 间接更新的单元格，即该 key（也就是 BE 对象）本身并没有具体填充数据而是因为父格扩展了才会得到扩展，该 value 中存储的 extendCellList 都是：拷贝的 BE 对象；那么，只需要遍历 key 的 childBEs 对象，从 beExtendBEsMap 中取出每个 childBE 子节点对应的 IndirectExtendedBEContainer 对象中存储拷贝单元格链表 childExtendBEList，将 extendCellList 和 childExtendBEList 一一对应即可，最后，如果该 key（也就是 BE 对象）是交叉单元格，上述操作只是更新了一个方向上的继承关系，因此，还要更新另一个方向上的继承关系；
9. 最后，遍历 cellElementExtendBEsMap，将每一个拷贝出来的 BE 对象链表置为对应单元格的 BEB 对象的 beList 属性；

上述过程对应的伪代码如下：

calCellElements	
Input:	单元格集合 elementCase
Output:	填充数据的表格矩阵 be_beb_2D
1:	isCalStateMatrix = new boolean[rowCount][columnCount];
	isCalculatingSet = new HashSet();
2:	For(elementCase 集合中的每个 cellElement)
3:	calCellElementAndSetState (cellElement);
4:	End





基于单元格扩展的报表引擎的研究与实现

第四章 单元格扩展原理

calCellElementAndSetState

Input: 抽象单元格对象 cellElement
表格矩阵 be_beb_2D

Output: 无

- 1: 计算该单元格对应的 BEB 对象 beB, 并放入 be_beb_2D 表格矩阵中;
- 2: 获取当前单元格 beB 对象包含的 BE 对象列表 beList;
- 3: 调用 refresh_be_array_relation(beList), 扩展、以该单元格为父格的继承链上的子格、更新继承关系;

refresh_be_array_relation

Input: 抽象单元格对应的 beList

Output: 无

- 1: cellElementExtendBEsMap = new HashMap();
- 2: For(beList 中的每个对象 be)
- 3: 调用 refresh_be_relation(be, cellElementExtendBEsMap) 扩展子格、更新继承关系;
- 4: End
- 5: 调用 refreshBEB(cellElementExtendBEsMap) 更新 BEB 对象的 beList 属性

refresh_be_relation

Input: be 对象
cellElementExtendBEsMap

Output: 无

- 1: beExtendBEsMap = new HashMap();
- 2: 将 <be, new DirectExtendedBEContainer(be.extendedBoxCELList)> 放入 beExtendBEsMap 中;
- 3: 获取 be.extendedBoxCELList 的长度 extendedCount;
- 4: For(be.childBEs 中的所有子格 childBE)
- 5: 将 childBE 拷贝出 extendedCount-1 份, 并将
<childBE, new InirectExtendedBEContainer(extendedCount 个 childBE)>
放入 beExtendBEsMap 中;
- 6: 获取 childBE 所在的抽象单元格 childCellElement, 将 <childCellElement,
extendedCount 个 childBE 组成的链表 > 放入 cellElementExtendBEsMap



```

-- -- -- -- 中;
7: -- -- -- 对于 childBE 的子格（即 be 的间接子格）也进行扩展，并将结果保存在 beExtendBEsMap 和 cellElementExtendBEsMap 中；
8: -- -- End
9: -- -- For(beExtendBEsMap 中的每个键值对<be, ExtendedBEContainer>)
10: -- -- 如果该键值对的 value 对象为 DirectExtendedBEContainer 类型，执行直接更新过程；
11: -- -- 否则，如果该键值对的 value 对象为 InirectExtendedBEContainer 类型，执行间接更新过程；
12: -- -- End

```

refreshBEB

```

Input:  cellElementExtendBEsMap
        be_beb_2D 矩阵
Output:  无
1:  For(cellElementExtendBEsMap 中的每个键值对<cellElement,list>)
2:  获取 cellElement 对应的 BEB 对象 beb;
3:  将 list 设置为 beb 对象的 childBEs 属性;
4:  End

```

4.6 计算最终结果矩阵

上一节详细描述了扩展子格、更新继承关系的过程。当所有的单元格都经过填数据、扩展子格、更新继承关系之后，我们会得到一个表格矩阵 be_beb_2D，该矩阵的行列大小和我们定义抽象工作表时的行列数是一样的。在这个矩阵中，每个位置都是一个 BEB 对象，每个 BEB 对象的 beList 属性中包含 n 个 BE 对象，每个 BE 对象的 extendedBoxCEList 属性中包含 n 个 NormalBoxCE 或 ExtendBoxCE 对象，保存具体的数据值。

所以，对于这个表格矩阵 be_beb_2D，一个抽象的单元格对应一个 BEB 对象，而一个 BEB 对象包含 n (n>=1) 个具体数据值。这种数据结构对于描述报表的结果很不方便的，也不利于最终的报表输出。所以，我们需要将 be_beb_2D 中的每个包含具体数据值的 NormalBoxCE 或 ExtendBoxCE 对象计算位置，放在它们在表格中的位置上。



1

2

3

4

5

6

1. 计算出单元格的深度序列 deepList;
2. 由浅入深遍历 deepList 中的每个单元格, 计算每个数据值在最终结果矩阵中的行列位置, 计算映射序列及最终结果矩阵;

4.6.1 计算单元格深度序列

由于单元格之间的继承属性, 所以子格的最终位置是依赖父格的, 只有父格放了正确的行列上, 子格才能根据父格的行列位置找到自己应该在的位置。

深度序列描述单元格在继承关系上的深度。没有父格的单元格在深度序列的第 0 层, 有一层父格的单元格在深度序列的第 1 层, 以此类推。计算最终结果矩阵要由浅入深遍历深度序列中的单元格。

单元格的深度计算过程如下:

1. 初始化一个空的 ArrayList 类型的 deeplist, 用于存储深度序列, 单元格在 ArrayList 中的位置就代表单元格的深度, 初始化一个空的 HashMap 类型的 map, 用于存储<FamilyMeber, deep>键值对, 遍历 genealogy 矩阵中的每一个 FamilyMember 对象, 执行 2;
2. 对于每一个 FamilyMeber 对象 familyMeber, 如果 map 中已包含该对象, 直接返回该对象对应的 deep 值, 否则, 转到 3;
3. 如果 familyMeber 有行父格, 那么对行父格 rowParent 执行 2, 得到行父格的深度 rowIndex, 如果 familyMeber 有列父格, 那么对列父格 rowParent 执行 2, 得到列父格的深度 columnIndex, 那么当前 familyMember 的深度 deepIndex=MAX(rowIndex, columnIndex)+1, 将<familyMeber, deepIndex>键值对放入 map 中, 将 familyMeber 对象放入 deeplist 的 deepIndex 位置上的 list (因为可能有多个单元格都在同一个深度上, 实际上, 这是很常见的), 返回当前单元格的深度 deepIndex;

根据上述过程, 对于年级成绩表计算后得到的 deepList 如下:

表 4-3 继承序列

0	A1,A3,C3,D3,A4,A5,D5
1	B4,C5
2	C4,D4

4.6.2 单元格定位顺序

1

2

3

4

5

6

该单元格在继承链上的深度。

对于在同一深度上的单元格，例如，深度为 0 的单元格有 A1、A3、C3、D3、A4、A5、D5，其中，A1、A3、D3、A5、D5 是不具有扩展属性的静态数据单元格，C3 是具有横向扩展属性的数据库单元格，A4 是具有纵向扩展属性的数据库单元格。此时，遵循这样的规则：同一深度的单元格，先定位具有扩展属性的单元格，再定位不具有扩展属性的单元格。这样做的原因是：如果先定位不具有扩展属性的单元格，结果矩阵当中，那么在定位具有扩展属性的单元格时，结果矩阵可能会增加行或列，此时之前定位的单元格的值就需要改变位置，那些值就要再被拷贝一次，放到正确位置上；反之，如果先定位具有扩展属性的单元格，再定位不具有扩展属性的单元格，就免除了上述的二次定位过程，可以提升整体的效率。

而对于不同深度的单元格，由于单元格之间的继承属性，所以子格的最终位置依赖父格的，只有父格放在了正确的行列上，子格才能根据父格的行列位置找到自应该在的位置。所以，应该遵循这样的规则：不同深度的单元格，先定位深度浅的单元格，再定位深度深的单元格。

4.6.3 最终结果对象：BoxCase

我们使用 BoxCase 对象来保存最终结果。该对象主要包括三个属性：

1. `int[]` 类型的 `rowMappingArray` 序列，用来保存抽象单元格与最终单元格之间的映射关系。例如，`rowMappingArray[5]=3` 表示在最终表格中处在第 5 行的数据在该单元格抽象定义时处在第 3 行，这个映射序列对后面的分页过程也有重要的作用；
2. `int[]` 类型的 `columnMappingArray` 序列，用来保存抽象单元格与最终单元格间列的映射关系；
3. `BoxCE[][]` 类型的 `boxCEMatrix` 矩阵，该矩阵一开始的行列数为报表抽象定义时的行列数，但是随着对深度序列中各个单元格的定位，该矩阵不断地被扩展，直到所有的单元格被放在正确的位置上为止；

BoxCase 对象保存了两个映射序列 `rowMappingArray` 和 `columnMappingArray` 序列，它保存了最终结果矩阵和抽象表矩阵之间的行列映射关系，这两个序列是数据定位的依据。如果一个 BE 对象具有扩展属性，那么当它所包含的 N ($N>1$) 个数据都被定位了之后，整个最终结果矩阵需要扩展，那么映射矩阵也会发生变化。





columnMappingArray 中的值为[0,1,2,3]，即最终矩阵的第 0 列对应抽象表格矩阵的第 0 列，以此类推。当当 C3 单元格的扩展值被定位之后，columnMappingArray 中的值要变成[0,1,2,2,2,2,2,3]，即最终矩阵的第 2 列-第 7 列都对应抽象表格矩阵的第 2 列。

4.6.4 继承区域对象：ExtendedArea

考虑一种情况，一个横向扩展的单元格所占大小为 1 行 1 列，它有一个下子格所占大小为 1 行 2 列，它还有一个下子格所占大小为 1 行 3 列，那么，如果该单元格行了数据库查询得到了 M (M>1) 个数据值，当该单元格的数据值对象被定位到最矩阵上去时，两两数据值之间应该相隔多少列呢？

这种情况如下图所示：

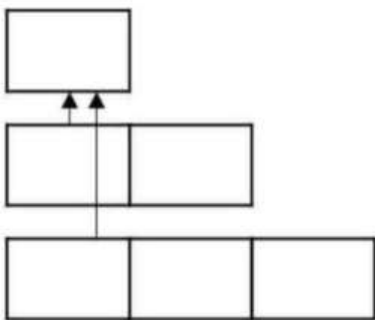


图 4-13 继承关系示意图

该单元格的数据值定位示意图如下图所示：

val1			val2			val3		

图 4-14 数据定位示意图

所以，仅仅有当前单元格的信息还是不够的，还需要知道以当前单元格为父格所有继承子格的情况，它们的大小会直接影响定位操作。

这里使用 ExtendedArea 对象来描述这种关系。ExtendedArea 对象描述的是以 BE 对象 be 为父格而得到的继承区域。ExtendedArea 对象需要保存以下信息：

- 1. column: be 在最终矩阵中的列；
- 2. row: be 在最终矩阵中的行；
- 3. fromColumn:继承区域在最终矩阵中的起始列；



1

2

3

4

5

6

5. toColumn:继承区域在最终矩阵中的结束列;
6. toRow:继承区域在最终矩阵中的结束行;
7. extendedBEList:be 执行数据库查询或计算后得到的数据值对象,是组成最终矩阵的基本元素;

计算某个 BE 对象 be 在最终结果矩阵中的行列位置分为两种情况: be 没有父格或 be 有父格。当 be 没有父格时,直接查找 rowMappingArray 序列或 columnMappingArray 序列,序列中 be 的行列所在的下标就是 be 在最终结果矩阵中的行列值;当 be 有父格时,根据单元格定位顺序,be 的父格肯定已经被定位过了,此时,需要根据父格所在的行列值判断 be 所在的行列值。

be 在最终结果矩阵中的列值计算过程如下:

1. 获取该 be 所在的抽象单元格的列值 index,如果 be 没有父节点,转到 2,否则转到 3;
2. 从 index 开始,向后遍历 columnMappingArray 序列,直到找到某个 columnMappingArray[i]=index,那么 i 就是 be 在最终结果矩阵中的列数,返回 i;
3. 获取 be 的父格在最终结果矩阵的列数 parentIndex,获取 columnMappingArray 序列在 parentIndex 位置上的值,也就是 be 的父格所在的抽象单元格的列 colIndex,如果 colIndex=index,说明父格和子格在定义时是出于同一列上,那么 parentIndex 就是 be 在最终结果矩阵上的列数,返回 parentIndex,否则转到 4;
4. 如果 index>colIndex,说明父格和子格在定义时父格所在的列比子格小,两格之间至少相差 index - colIndex 列,转到 5;如果 index<colIndex,说明父格子格在定义时父格所在的列比子格大,两格之间至少相差 colIndex - index 列,转到 6;
5. 此时从 parentIndex + (index - colIndex)处向后遍历 columnMappingArray 序列,直到找到某个 columnMappingArray[i] = index,那么 i 就是 be 在最终结果矩阵中的列数,返回 i;
6. 此时从 parentIndex - (colIndex - index)处向前遍历 columnMappingArray 序列,直到找到某个 columnMappingArray[i] = index,那么 i 就是 be 在最终结果矩阵中的列数,返回 i;





以上述情形为例，具体的 ExtendedArea 对象的计算过程如下：

假设上述情形对应的抽象单元格如下所示（为了简单起见，假设 B2 单元格没有子格）：

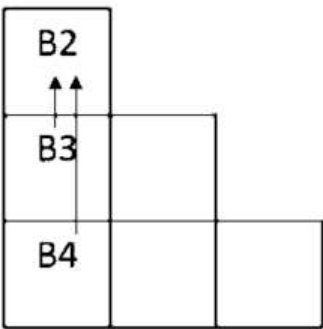


图 4-15 继承关系示意图

那么在抽象定义上，该 ExtendedArea 的起始行（行数从 0 开始计数）为 1，束行为 4，起始列（列数从 A 开始计数）为 1，结束列为 4：

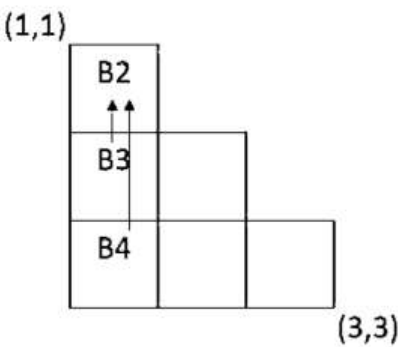


图 4-16 继承区域范围示意图

假设现在有 rowMappingArray 如下所示：

0	0	1	2	3	4	...
---	---	---	---	---	---	-----

图 4-17 初始行映射序列示意图

columnMappingArray 如下所示：

0	0	0	1	2	3	4	...
---	---	---	---	---	---	---	-----

图 4-18 初始列映射序列示意图

根据 ExtendedArea 的抽象区域位置，查询映射序列，ExtendedArea 对象的各个属性如下：

- 1. column : 3;
- 2. row : 2;





- 3. fromColumn : 3;
- 4. fromRow : 2;
- 5. toColumn : 5;
- 6. toRow : 4;

即，该 ExtendedArea 在最终结果矩阵中的位置如下所示：

							...
							...
							...
							...
							...
							...
...	...	...	...	...	...	...	...

图 4-19 继承区域在最终结果矩阵中的位置示意图

根据以上描述，我们可以得到，横向扩展的父格在最终矩阵中定位时，两两数值之间应该间隔 toColumn- fromColumn+1 个列，纵向扩展的父格在最终矩阵中定位时，两两数据值之间应该间隔 toRow - fromRow +1 个行。如果父格横向扩展，定位个父格的所有数据值对象时，最终结果矩阵总共需要增加（toColumn- fromColumn+1）*（extendedBEList.size -1）列，如果父格纵向扩展，定位整个父格所有数据值对象时，最终结果矩阵总共需要增加（toRow - fromRow +1）*（extendedBEList.size -1）行。

4.6.5 设置数值单元格的最终位置

经过上述小节的描述，我们知道了应该由浅入深、由扩展到非扩展的顺序，定位单元格。并且，在定位过程中，需要借助 ExtendedArea 对象才能确定数据值位置的增和最终结果矩阵每次应该增加的行数或列数。ExtendedArea 对象描述了一个 BE 对象继承区域，一个 ExtendedArea 对象对应一个 BE 对象。

在向最终结果矩阵设置数据值之前，需要知道每个数据值的最终位置。设置数





1. 获取以前的 ExtendedArea 对象定位增加的行数或列数 offset，获取 ExtendedArea 对象的起始位置 fromIndex（如果 BE 所在的单元格横向扩展那么获取起始列，如果纵向扩展，那么获取起始行）、结束位置 toIndex（如果 BE 所在的单元格横向扩展，那么获取结束列，如果纵向扩展，那么获取结束行）、单元格之间的行列偏移量 offsetBetweenBoxCEs（计算公式为 $\text{toIndex} - \text{fromIndex} + 1$ ）、数据值个数 extendedBELength、此次定位会给最终结果矩阵增加的行数或列数 addedBoxCECount（计算公式为 $\text{offsetBetweenBoxCEs} * (\text{extendedBELength} - 1)$ ），初始化下个单元格的总偏移量 nextIndex = 0；
2. 遍历 ExtendedArea 对象 extendedBEList 属性中的每一个具体数据值 boxCE 设置 boxCE 在最终结果矩阵上的最终行列位置，转到 3，遍历结束，转到 3；
3. 如果 BE 横向扩展，那么 boxCE 的行等于 ExtendedArea 对象的 row 属性，boxCE 的列等于 ExtendedArea 对象的 column + nextIndex + offset，将 nextIndex 加上 offsetBetweenBoxCEs 偏移量，转到 2；
4. 如果 BE 纵向扩展，那么 boxCE 的列等于 ExtendedArea 对象的 column 属性，boxCE 的行等于 ExtendedArea 对象的 row + nextIndex + offset，将 nextIndex 加上 offsetBetweenBoxCEs 偏移量，转到 2；
5. 返回此次定位会给最终结果矩阵增加的行数或列数 addedBoxCECount

4.6.6 重新计算映射序列

BoxCase 对象保存了两个映射序列 rowMappingArray 和 columnMappingArray 序列，它保存了最终结果矩阵和抽象单元格矩阵之间的行列映射关系。例如，rowMappingArray[5]=3 表示在最终表格中处在第 5 行的数据在该抽象表格矩阵中处在第 3 行。这两个序列是数据值定位的依据。如果一个 BE 对象具有扩展属性，那么所包含的 $M(M>1)$ 个数据值都被定位了之后，整个最终结果矩阵需要扩展，那么映射序列也会发生变化。

如果有一个 columnMappingArray，抽象单元格矩阵中 x 列在映射序列中的位置如下图所示：

0	1	2	...	x	...	a	b	c
---	---	---	-----	---	-----	---	---	---

图 4-20 x 列在映射序列中的位置示意图

那么，如果某个位于 x 列的单元格扩展出来 k 列，这个序列就要变成如下图所示



示：

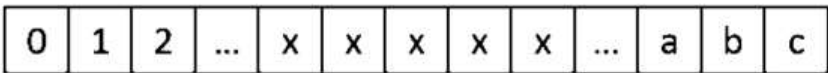


图 4-21 映射序列更新示意图

该序列的更新原理如下所示：

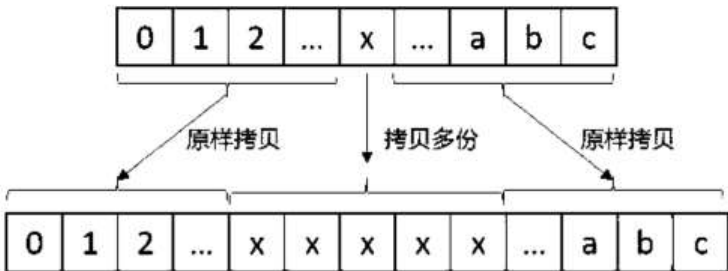


图 4-22 映射序列更新原理示意图

上面的情况是最简单的一种情况，还有一种稍微复杂的情况是，抽象单元格矩中 x 列、 $y(y=x+1)$ 列在映射序列中的位置如下图所示：

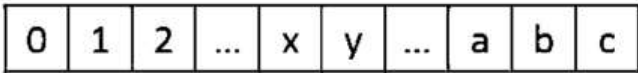


图 4-23 映射序列更新原理示意图

那么，如果某个位于 x 列的单元格扩展出来 3 列，而这个单元格有个子格占据列、 y 列两列，那么这个序列就要变成如下图所示：

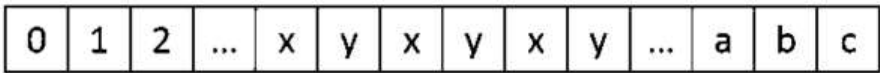


图 4-24 复杂情况下映射序列更新示意图

该序列的变换原理如下所示：



图 4-25 复杂情况下映射序列更新原理示意图

由上图得知，如果要重新计算映射序列，需要以下信息：

- 1. startPos:映射序列进行复制的起始位置；
- 2. fromIndex:映射序列需要复制的区域的起始位置；



4. copyLength:映射需要复制的区域的个数;

上图对应的上述信息如下图所示:

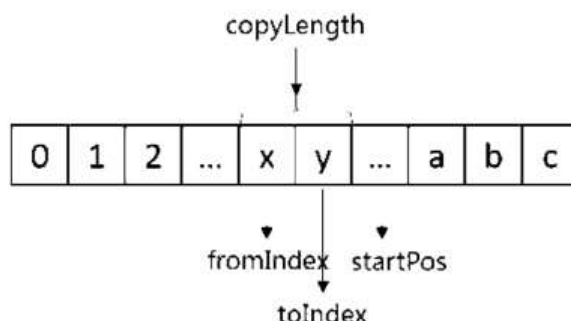


图 4-26 映射序列更新示意图

以上所需的信息保存在一个长度为 4 的 `int[]` 类型的数组 `info` 当中。在上一小节置数值单元格的最终位置中可以得到这些数组信息。由于在设置数值单元格的最终位置时，可能一次设置多个 BE 对象的扩展格的位置，所以，实际上得到一个保存有 `info` 的链表 `infoList`。

更新计算映射序列的过程如下所示:

1. 计算本次扩展会增加的总行数或总列数 `addedCount`: 对于每个 `info`, 增加的行数或列数为 $(toIndex - fromIndex) * copyLength$, 遍历 `infoList`, 得到这些行数或列数的和, 就是本次扩展会增加的总行数或总列数 `addedCount`, 转到 2;
2. 将 `columnMappingArray` 拷贝到 `columnMappingArrayCopy` 中, 然后将 `columnMappingArray` 扩充到原本的长度+`addedCount` 大小, 令 `srcPos=0`、`destPos=0`, 执行 3;
3. 遍历每一个 `info`, 执行 4, 遍历完成后, 执行 6;
4. 先将 `columnMappingArrayCopy` 中 `srcPos` 开始的 `startPos-srcPos` 个数据拷贝到 `columnMappingArray` 中 `destPos` 开始的 `startPos-srcPos` 个位置上, 这些是需要复制的区域之前的数据, 执行 4;
5. 然后执行拷贝多份的操作, 令 `srcPos=startPos`、`destPos += startPos`, 将 `columnMappingArrayCopy` 中 `fromIndex` 开始到 `toIndex` 结束的数据拷贝到 `columnMappingArraydestPos` 开始的 `toIndex-fromIndex+1` 个位置上, 这个拷贝需要执行 `copyLength` 次;
6. 最后将 `columnMappingArrayCopy` 中剩下的数据拷贝到 `columnMappingArray` 中即可;





4.6.7 计算最终结果矩阵

最终结果矩阵的计算和映射序列的计算相似，但是也有微小的不同：映射序列扩展和设置是同时进行的，而最终结果矩阵 boxCEMatrix 是先扩展，然后再设置数值。

以年级成绩表为例，根据深度序列和单元格的定位顺序，先对 C3、A4 这两个有扩展属性的单元格的数据值进行定位，再对 A1、A3、D3、A5、D5 这 5 个不具有扩展属性的单元格的数据值进行定位，再对深度为 1 并且具有扩展属性的 B4 单元格的数据值进行定位，再对深度为 1 不具有扩展属性的 C5 单元格的数据值进行定位，最后对深度为 2 不具有扩展属性的 C4、D4 单元格的数据值进行定位。

以扩充 boxCEMatrix 的列为例，最终结果矩阵的计算过程如下所示：

1. 计算本次扩展会增加的总列数 addedCount：对于每个 info，增加的列数为 $(toIndex - fromIndex) * copyLength$ ，遍历 infoList，得到这些列数的和，就是本次扩展会增加的总列数 addedCount，转到 2；
2. 将 boxCEMatrix 拷贝到 boxCEMatrixCopy 中，然后将 boxCEMatrix 的每行列数扩充到原本的长度+addedCount 大小，执行 3；
3. 遍历 boxCEMatrix 的每一行，执行 4；
4. 遍历 infoList 中的每个 info，执行 5，遍历完成后，执行 7；
5. 令 srcPos=0、destPos=0，将 boxCEMatrixCopy 中该行位于 info 的 startIndex 前的数据值对象拷贝到 boxCEMatrix 中，如果数据值对象不为空，重置该数据值对象的列数；
6. destCol 增加 $copyLength * (toIndex - fromIndex + 1)$ ；
7. 复制 boxCEMatrixCopy 中该行剩下的数据值对象到 boxCEMatrix 中，如果数据值对象不为 null，重新设置该数据值对象的列数；





第五章 工作簿分页与数据展示

单元格的扩展原理章节详细讲述了一个抽象定义的表格如何扩展成为一个具体二维结果矩阵的过程。如果数据库中的数据数量过多，那么得到的二维矩阵的行或列就会很大，而最终生成的页面的高和宽是固定的（例如，A4 纸大小等）。此时，就要对得到的结果矩阵分页展示。

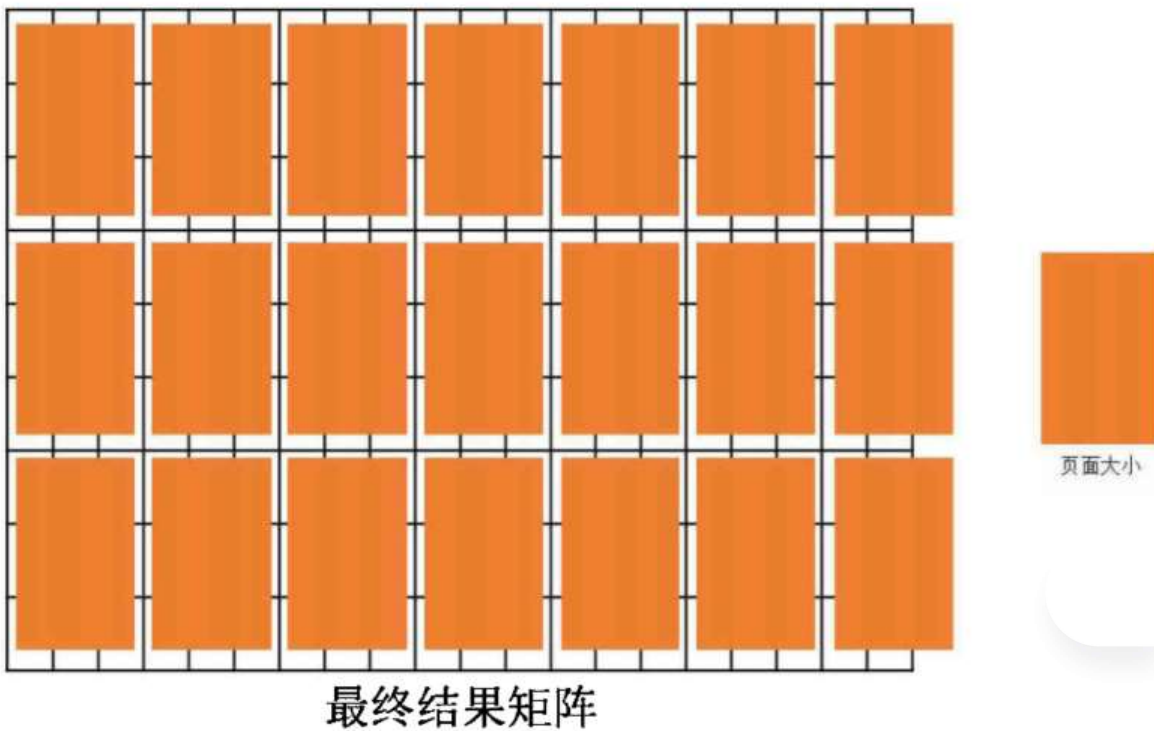


图 5-1 结果矩阵分页示意图

而且，一个工作簿节点（<WorkBook>标签）包含多个工作表节点（<Report>标签）。不同的工作表也需要在不同的页进行展示。

工作簿的分页过程如下所示：

- 1. 遍历工作簿的每个工作表，获取纸张的可用宽高；
- 2. 获取工作表中的行前分页、行后分页、列前分页、列后分页信息；
- 3. 如果工作表中没有用到重复标题行或重复结尾行，执行工作表的生成过程；
- 4. 如果工作表用有用到重复标题行或重复结尾行，执行工作表的有重复元素的生成过程；
- 5. 根据工作表的分页信息，生成相应的分页对象，并将分页对象放入该工作表对应的页面集合中；





- 6. 遍历完成，得到该工作簿所有工作表生成的所有页面；
- 本章以地区订单明细表详细讲述每一步的过程。

5.1 影响分页的因素

工作表的分页主要受以下 4 个方面的因素的影响：

- 1. 纸张去除页边距和页眉页脚之后的可用宽高；
- 2. 每个单元格的大小；
- 3. 工作表是否存在重复标题行、重复结尾行；
- 4. 某一单元格是否具有行前分页、行后分页、列前分页、列后分页属性；

5.1.1 纸张可用宽高

纸张的可用宽高区域如下图所示：

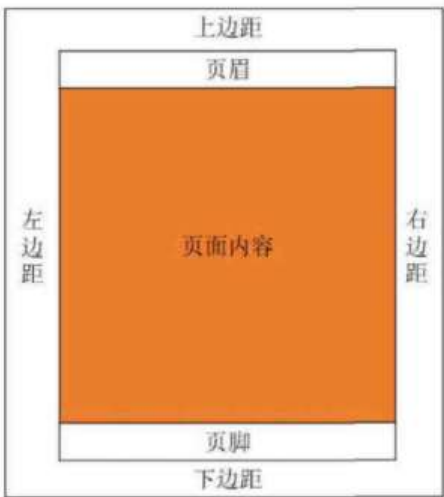


图 5-2 纸张可用区域示意图

5.1.2 单元格大小

在定义单元格时，每个单元格都可能有不同的宽高，而纸张的可用大小是固定的，因此，每个单元格的大小也会影响分页的结果。

5.1.3 重复行列

如果工作表存在重复标题行、重复结尾行、重复标题行、重复结尾行，那么每页中除了展示数据外，还要给重复行列预留一定的宽度和高度。







图 5-3 重复行列示意图

5.1.4 行列分页属性

如果某单元格设置了行列分页属性，那么，即使当前页并没有用完，也要进行页显示下面的数据。

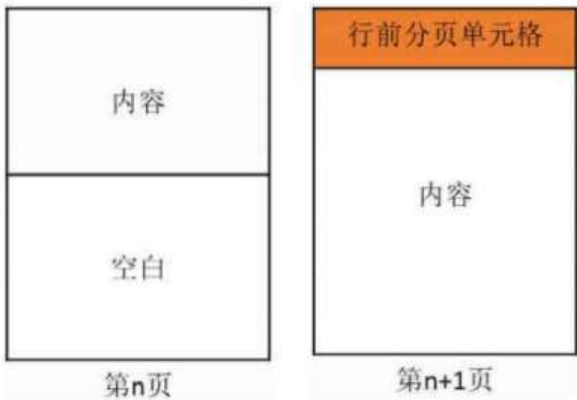


图 5-4 行列分页属性示意图

5.2 获取工作表的行列分页信息

根据上一节的描述，单元格的行列分页属性会影响工作表的分页效果。因此，页过程的第一步就是要获取工作表的行列分页信息。

获取工作表的行列分页信息过程如下：

1. 创建 int 型链表 pageByRowList、pageByColList，遍历最终结果矩阵的每一单元格，执行 2；
2. 如果当前单元格有行前分页属性，将该单元格所在的行 row 放入 pageByRowList 中；
3. 如果当前单元格有行后分页属性，将该单元格所在的行 row + 该单元格合的行数 rowSpan 放入 pageByRowList 中；
4. 如果当前单元格有列前分页属性，将该单元格所在的列 column 放入





- 5. 如果当前单元格有列后分页属性，将该单元格所在的列 `column` + 该单元格并的行数 `columnSpan` 放入 `pageByColList` 中；

5.3 生成页面分布序列

根据上面章节的描述，工作表的重复行列属性会影响工作表的分页效果。

对于没有重复行列属性的工作表，工作表的页面生成只要沿着最终结果矩阵从到右、从上到下依次生成即可。这种分页过程叫普通分页过程。

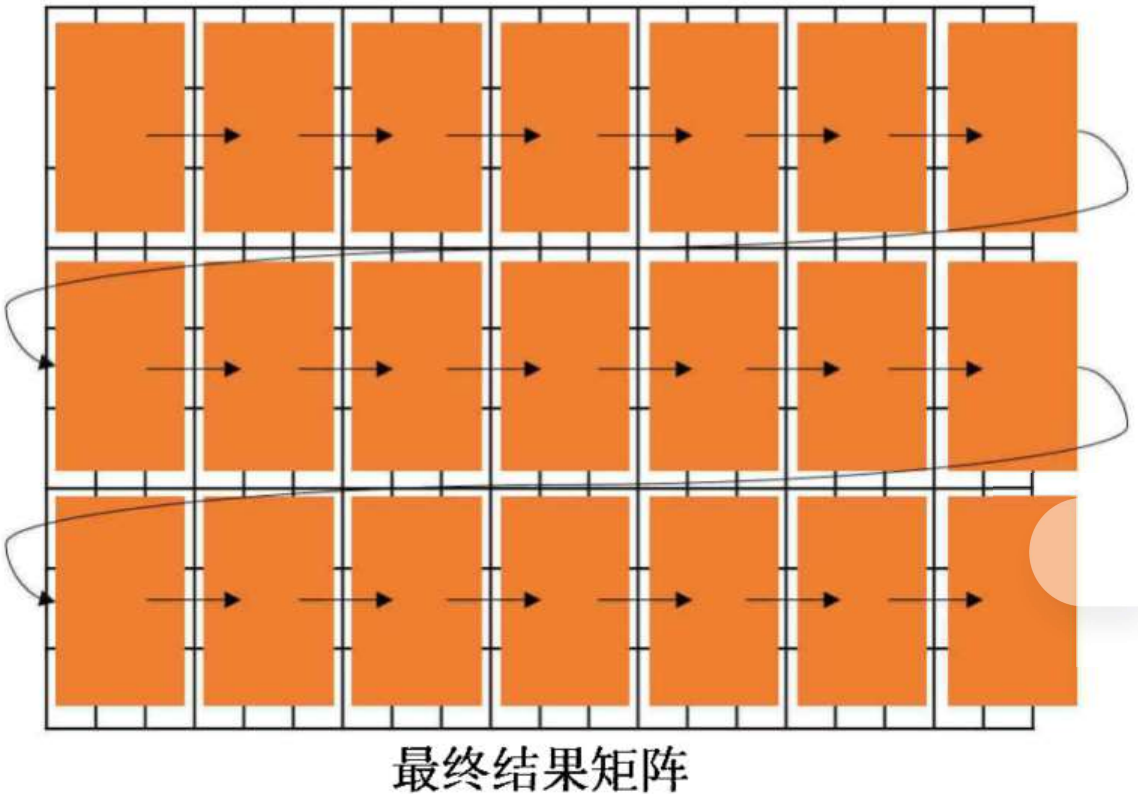


图 5-5 普通分页意图

而对于有重复行列属性的工作表，工作表的页面生成过程不仅要考虑应该展示数据，还要在页面的相应位置加上重复的标题和结尾元素，不再是简单的沿着最终果矩阵从左到右、从上到下依次生成。这种分页过程叫做重复分页过程。





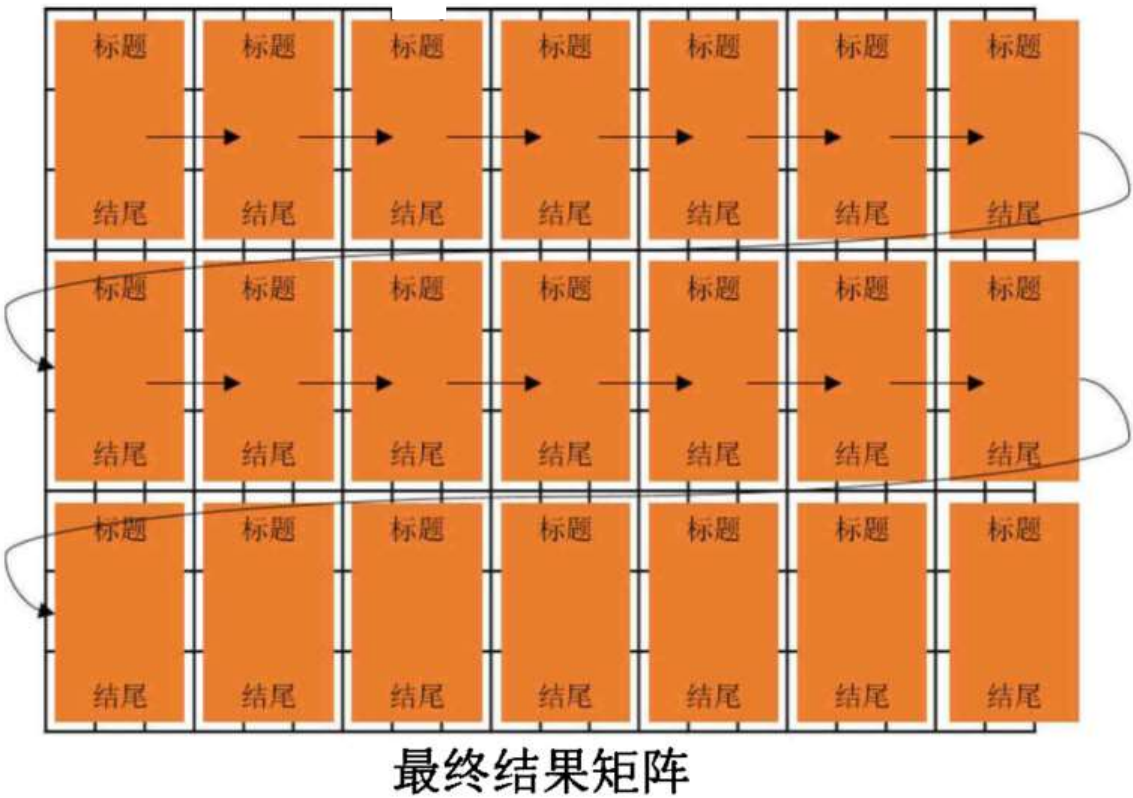
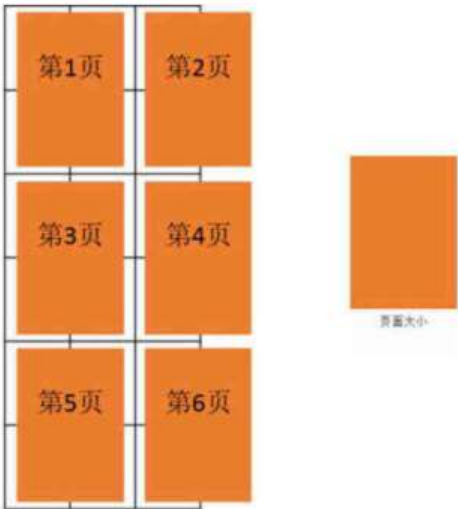


图 5-6 重复分页意图

5.3.1 区域分布序列

页面区域分布序列是描述每个工作表在行分布或列分布上区域信息的序列。一工作表有两个页面分布序列，pageRowIndexList 和 pageColumnIndexList。pageRowIndexList 对应该工作表在行分布上的区域信息，pageColumnIndexList 对应工作表在列分布上的区域信息。

例如，一个工作表对应的最终结果矩阵有 6 行 3 列。假设每页只能显示 2 行 2 的数据，那么该工作表最终被分为 6 页。如下所示：





基于单元格扩展的报表引擎的研究与实现



那么，在列分布上，该工作表被划分为两个区域，在行分布上，该工作表被划分为 3 个区域。页面区域分布序列就是用来描述这些区域信息的。

一个区域信息用一个 IdxHeaderFooter 对象表示。一个 IdxHeaderFooter 有 3 个属性用来描述区域信息：

- 1. Idx：最终结果矩阵中该区域的开始行或开始列；
 - 2. headerRepeatList：int 型链表，存储该区域的重复标题元素的行列信息；
 - 3. footerRepeatList：int 型链表，存储该区域的重复结尾元素的行列信息；
- 区域信息和分页内容之间的对应如下所示：

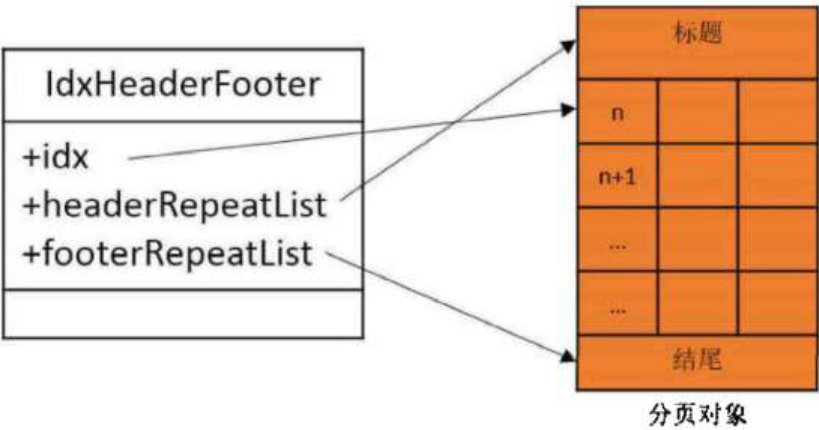


图 5-8 区域信息属性-分页内容对照图

根据上述分析，一个工作表的总页数 $pageCount = \text{行分布区域数} \times \text{列分布区域数}$

5.3.2 普通分页

如果工作表中没有重复元素，那么该工作表的分页执行普通分页过程。

在行分布下，这种情况下生成页面分布序列 `pageRowIndexList` 的具体过程如下

- 1. 第一个区域开始，创建一个 `IdxHeaderFooter` 对象，该对象的 `idx` 属性为 0，该对象放入 `pageRowIndexList` 中，初始化页面的总高度 `totalRowHeight=0`；
- 2. 遍历最终结果矩阵的每一行，执行 3，遍历结束后，执行 6；
- 3. `totalRowHeight += 当前行的高度`，如果 `totalRowHeight > 页面的可用高度`，或当前行在 `pageByRowList` 中，则需要分区，转到 4，否则，继续遍历下一行；
- 4. 如果 `pageRowIndexList` 中的最后一个区域信息的 `idx` 等于当前行，或当前的行高 $>$ 页面的可用高度，终止分页过程，抛出死循环异常，否则转到 5；
- 5. 创建一个 `IdxHeaderFooter` 对象，该对象的 `idx` 属性为当前行数，将该对象放入 `pageRowIndexList` 中，标志新的一个区域开始，`totalRowHeight` 重置为当前行的高度。





6. 创建一个 IdxHeaderFooter 对象，该对象的 idx 属性为最终结果矩阵的最大数，将该对象放入 pageRowIndexList 中，标志最后一个区域的结束；
上述过程对应的伪代码如下所示：

dealWithRowBreak	
Input:	最终结果矩阵的最大行数 maxRow 最终结果矩阵的单元格高度序列 rowHeightList 页面的可用高度 pageHeight 行前分页信息 pageByRowList
Output:	行分布上的区域分布序列 pageRowIndexList
1:	start = new IdxHeaderFooter(0); pageRowIndexList.add(start); totalRowHeight = 0;
2:	For(int currentRow = 0; currentRow < maxRow; i++)
3:	获取当前行的高度 rowHeight = rowHeightList[currentRow]
4:	totalRowHeight += rowHeight;
5:	If(totalRowHeight > pageHeight pageByRowList 包含该行)
6:	If(pageRowIndexList.last.idx = currentRow rowHeight > pageHeight)
7:	抛出死循环异常;
8:	pageRowIndexList.add(new IdxHeaderFooter(currentRow));
9:	totalRowHeight = rowHeight;
10:	End
11:	pageRowIndexList.add(new IdxHeaderFooter(maxRow)), 标志最后一个区域的结束;

列分布下生成区域分布序列 pageColumnIndexList 的过程与上述行分布下生成 pageRowIndexList 的过程相似，此处不再赘述。

5.3.3 有重复元素的分页

如果工作表中有重复元素，那么该工作表的分页执行有重复元素的分页过程。这种情况下生成页面分布序列 pageColumnIndexList 的具体过程如下：

1. 初始化 repeatFooterMappingRowList、repeatHeaderMappingRowList 、repeatFooterRowList，初始化本页的数据总高度 totalHeight=0，转到 2；
2. 计算第一个区域的重复结尾行的高度 repeatFooterHeight，将其加到 totalHe

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

- 上，并将重复结尾行信息放入 repeatFooterRowList 中，创建一个 IdxHeaderFooter 对象，该对象的 idx 属性值为 0，footerRepeatList 属性值为 repeatFooterRowList，将该对象放入 pageRowIndexList 中，转到 3；
3. 遍历每一行，执行 4，遍历完成后，执行 14；
 4. 如果当前行属于重复结尾行并且不在 pageByRowList 中，将当前行信息从 pageRowIndexList 的最后一个区域对象的 footerRepeatList 属性中移除，继续遍历下一行，否则，执行 5；
 5. 获取当前行的高度，加到 totalHeight 上，初始化分页标志 shouldPaginate=false，如果 totalHeight>paperHeight 或者该行需要行前分页，转到 6，否则，转到 7；
 6. 如果 pageRowIndexList 的最后一个区域对象的 idx 属性等于当前行，说明出现了分区死循环，停止生成页面分布序列，抛出死循环异常，否则，令 shouldPaginate=true，转到 7；
 7. 如果当前行及后面的所有行都是重复结尾行，从 pageRowIndexList 的最后一个区域信息中去掉这些行信息，说明已经到了最终结果矩阵的最后，无需分页，转到 14，否则，转到 8；
 8. 如果 shouldPaginate=false，说明此时不需要分区，继续遍历下一行，否则，如果 shouldPaginate=true，说明需要分区，执行 9；
 9. 清空 repeatHeaderMappingRowList 和 repeatFooterMappingRowList，初始化 totalRepeatHeight，用来记录重复标题的总高度，将当前行保存到 currentRowCopy 中，转到 10；
 10. 如果当前行不是最后一行并且当前行是重复标题行，则将当前行的高度加到 totalHeight 上，将当前行对应的抽象行放入 repeatHeaderMappingRowList 中，递增当前行直到当前行是最后一行或者当前行不是重复标题行，转到 11；
 11. 如果当前行是最后一行，那么该区域是最后一个区域，创建一个 IdxHeaderFooter 对象，该对象的 idx 属性值为 currentRowCopy，将该对象放入 pageRowIndexList 中，转到 14；
 12. 初始化 headerRepeatList、footerRepeatList，用来存放下一个区域的重复标题行、重复结尾行信息，转到 13；

基于单元格扩展的报表引擎的研究与实现

1

2

3

4

5

6

基于单元格扩展的报表引擎的研究与实现

第五章 工作簿分页与数据展示

的高度加到 totalHeight 上；得到下一区域的重复结尾行信息，放入 footerRepeatList 中，并将重复结尾行的高度加到 totalHeight 上；创建一个 IdxHeaderFooter 对象，该对象的 idx 属性值为 currentRowCopy，该对象的为 headerRepeatList，该对象的属性为 footerRepeatList，将该对象放入 pageRowIndexList 中，将 totalHeight 重置为当前行高度，继续下一行遍历；

14. 创建一个 IdxHeaderFooter 对象，该对象的 idx 属性为最终结果矩阵的最大数，将该对象放入 pageRowIndexList 中，标志最后一个区域的结束；

上述过程对应的伪代码如下所示：

paginate_according_to_repeat	
Input:	最终结果矩阵的最大行数 maxRow 最终结果矩阵的单元格高度序列 rowHeightList 最终结果矩阵的行映射序列 rowMappingArray 页面的可用高度 pageHeight 行前分页信息 pageByRowList 重复标题的开始行 repeatHeaderFrom 重复标题的结束行 repeatHeaderTo 重复结尾的开始行 repeatFooterFrom 重复结尾的结束行 repeatFooterTo
Output:	行分布上的区域分布序列 pageRowIndexList
1:	repeatFooterMappingRowList repeatHeaderMappingRowList repeatFooterRowList totalHeight=0
2:	计算第一个区域的重复结尾行的高度 repeatFooterHeight， totalHeight+=repeatFooterHeight，将重复行信息放入 repeatFooterRowList 中；
3:	For(int currentRow =0; currentRow<maxRow;i++)
4:	If(currentRow 属于重复结尾行&¤tRow 不在 pageByRowList 中)
5:	从 pageRowIndexList 的最后一个区域信息的 footerRepeatList 中 移除 currentRow；
6:	Else
7:	获取当前行的高度 rowHeight = rowHeightList[currentRow]
8:	totalHeight += rowHeight;



1

2

3

4

5

55



```
-----
                                headerRepeatList, footerRepeatList);
                                pageRowIndexList.add(next);
31: ----- 将 totalHeight 重置为当前行高 rowHeight;
32: ----- End
33: ----- pageRowIndexList.add(new IdxHeaderFooter(maxRow)), 标志最后一个区域
                                的结束;
```

列分布下生成区域分布序列 pageColumnIndexList 的过程与上述行分布下生成 pageRowIndexList 的过程相似，此处不再赘述。

假设订单表中华北地区有 70 个订单，华东地区有 20 个订单，得到的最终结果阵如下所示：

第0行	华北			
70条数据	订单ID	客户ID	订购日期	应付金额
第71行	华东			
20条数据	订单ID	客户ID	订购日期	应付金额
第92行	地区订单明细表			

图 5-9 最终结果矩阵示意图

假设报表的设计中，地区所在的单元格设置了行前分页，最后一行设置了重复尾行。为简单起见，假设每页能够展示 50 行 10 列大小的信息。那么，根据上述过程，pageRowIndexList 的生成过程如下所示：

- 1. 生成第 1 个区域的 IdxHeaderFooter 对象 area1，idx 属性为 0，第 0 行为重复行，但是由于第 0 行位于本区域的范围内，所以不会将第 0 行的行索引放入 area1 对象的 headerRepeatList 属性中，由于该工作表有重复结尾行，所以将第 92 行的行索引放入 area1 对象的 footerRepeatList 属性中；

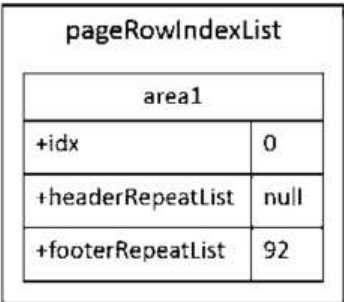


图 5-10 area1 示意图

- 2. 由于每页只能放 50 行信息，所以遍历到第 49 行时，第 1 页已经达到页面



上限了（0-48 行和重复结尾行），所以第 49 行要放在新的一页。生成第 2 个区域的 IdxHeaderFooter 对象 area2，idx 属性为 49，从当前行向前寻找重复标题行，然后把重复标题行的行号 0 放入 area2 对象的 headerRepeatList 属性中，将第 92 行的行索引放入 area1 对象的 footerRepeatList 属性中；

pageRowIndexList			
area1		area2	
+idx	0	+idx	49
+headerRepeatList	null	+headerRepeatList	0
+footerRepeatList	92	+footerRepeatList	92

图 5-11 area2 示意图

3. 当遍历到第 71 行时，虽然该区域对应的页面还没有装满 50 条数据，但是第 71 行设置了行前分页属性，所以第 71 行要放在新的一页。生成第 3 个区域的 IdxHeaderFooter 对象 area3，idx 属性为 71，但是由于第 71 行位于本区域范围内，所以不会将第 71 行的行索引放入 area3 对象的 headerRepeatList 属性中，同理，第 92 行也在本区域内，所以不会将第 92 行的行索引放入 area1 对象的 footerRepeatList 属性中；

pageRowIndexList					
area1		area2		area3	
+idx	0	+idx	49	+idx	71
+headerRepeatList	null	+headerRepeatList	0	+headerRepeatList	null
+footerRepeatList	92	+footerRepeatList	92	+footerRepeatList	null

图 5-12 area3 示意图

4. 创建最后一个区域的 IdxHeaderFooter 对象 area4，该对象的 idx 属性为最终结果矩阵的最大行数 92，将该对象放入 pageRowIndexList 中，标志着一个区域的结束；

pageRowIndexList							
area1		area2		area3		area4	
+idx	0	+idx	49	+idx	71	+idx	93
+headerRepeatList	null	+headerRepeatList	0	+headerRepeatList	null	+headerRepeatList	null
+footerRepeatList	92	+footerRepeatList	92	+footerRepeatList	null	+footerRepeatList	null





pageRowIndexList 的生成相对就简单很多，因为一个页面可以放 10 列数据，而一个工作表只有 4 列数据，所以 pageRowIndexList 中只有两个 IdxHeaderFooter：

pageColumnIndexList			
area1		area2	
+idx	0	+idx	4
+headerRepeatList	null	+headerRepeatList	null
+footerRepeatList	null	+footerRepeatList	null

图 5-14 pageRowIndexList 示意图

所以，该工作表对应的最终结果矩阵在行分布上被划分为 3 个区域，在列分布上划分为 1 个区域，该工作表最终会生成 3*1=3 页报表。

5.4 生成分页序列

上一节详细讲述了区域分布序列的生成过程。区域分布序列只是将最终结果矩阵在行列分布上划分为若干个区域，每个区域对应一个分页范围。这一节会根据区域分布序列生成工作表的分页序列。

假设某个最终结果矩阵有 14 行 14 列，每页可以显示 5 行 4 列的信息，而且该工作表执行普通分页过程。那么该工作表生成的区域分布序列如下所示：

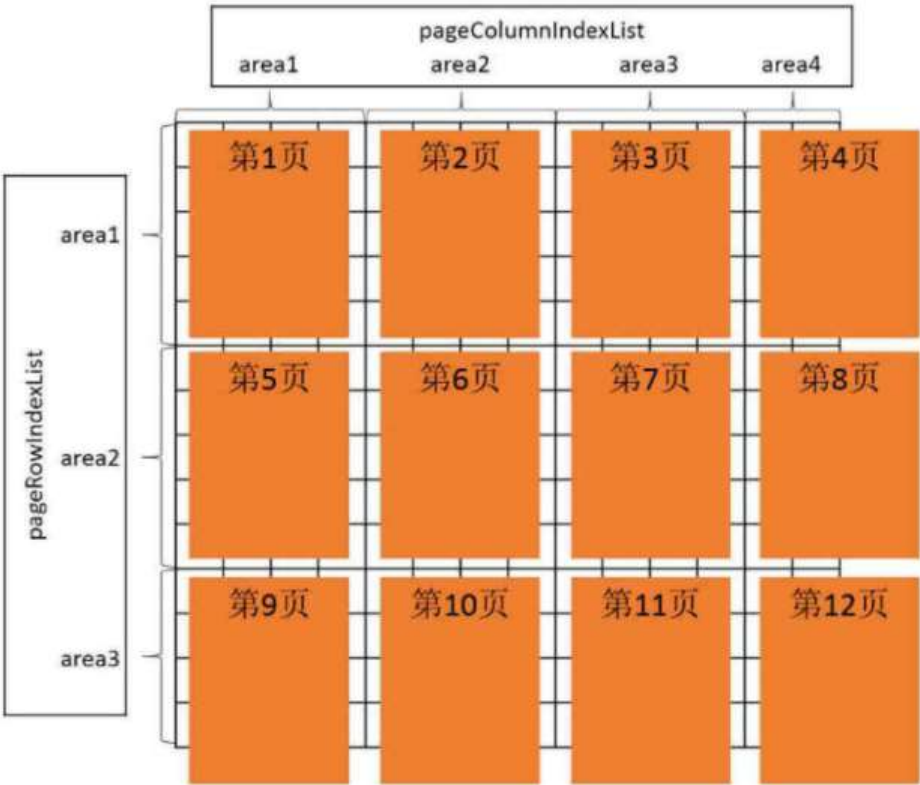


图 5-15 区域分布序列示意图





0-4 行、0-3 列的数据，第二页显示 0-4 行、4-7 列的数据，以此类推。

如果一个工作簿有重复标题行、重复结尾行、重复标题列或重复结尾列，那么工作簿的每个分页的包含的行列区域可能有多个。例如，某个工作簿行分布上的对象如下所示：

area n	
+idx	100
+headerRepeatList	[0,1,2,3]
+footerRepeatList	[200]

area n+1	
+idx	150
+headerRepeatList	[0,1,2,3]
+footerRepeatList	[200]

图 5-16 区域分布序列示意图

那么，第 n 区域对应的分页包含的数据行范围为[0-4]、[100-149]、[200]。

分页对象需要具体描述了每一页的行列详细信息。分页对象用表示 ClippedECI 类表示。该类有 9 个与页面信息有关的属性：

- 1. source_element_case: 保存对最终结果矩阵的引用；
- 2. row_line_array: 该分页的包含的数据的行范围列表；
- 3. column_line_array: 该分页的包含的数据的列范围列表；
- 4. rowHeightList: 该工作表的行高列表；
- 5. columnWidthList: 该工作表的列宽列表；
- 6. row_height_index_list: 该分页包含的行索引列表；
- 7. column_width_index_list: 该分页包含的列索引列表；
- 8. paperSetting: 该分页的纸张设置；
- 9. totalPageNumber: 该报表的总分页数；

5.5 数据展示

上面的章节详细讲述了数据的分页过程。将最终结果矩阵分为多个页面角色之后，就需要对分页进行展示。

本节介绍两张数据展示方式：HTML 页面展示方式、EXCEL 文档展示方式。

5.5.1 HTML 页面展示

根据最终结果矩阵生成 HTML 标签需要将最终结果矩阵中的数据与 HTML 中的 table 子标签一一对应。

假设现在有一个数据矩阵如下所示。



员工ID	员工姓名	员工薪水
19	Shuang	1000
18	Daisy	5500
17	张三	1000
16	秦可卿	6000
15	贾宝玉	3000
12	林黛玉	3000
11	路飞	1500
9	张三	1000
8	李四	2000
7	张三	1000
6	李四	2000
5	张三	1000
4	李四	2000
3	张三	1000

图 5-17 分页数据矩阵

那么，应该生成如下的 HTML 数据：

```
<table>
  <tbody>
    <tr>
      <td>员工 ID</td>
      <td>员工姓名</td>
      <td>员工薪水</td>
    </tr>
    <tr>
      <td>19</td>
      <td>Shuang</td>
      <td>1000</td>
    </tr>
    <tr>
      <td>18</td>
      <td>Daisy</td>
      <td>5500</td>
    </tr>
    .....
  </tbody>
```





<table>

由上可见，生成的 HTML 数据有两种：一般标签和数据内容。在上述 HTML 数据中，<table>、<tbody>、<tr>、<td>都属于一般标签，这种标签能够拥有子标签、兄弟标签或数据内容；而具体的数据，例如“员工 ID”、“员工姓名”、“员工薪水”等都属数据内容，只能作为一般标签的内容存在。

根据上述分析，HTML 数据结构设计如下：

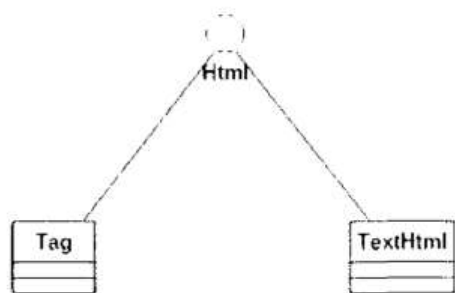


图 5-18 HTML 类图

其中，Tag 类代表一般标签，它有以下属性：

1. tagName: String 类型，存储标签名；
2. id: List 类型，存储标签的 ID 属性；
3. classes: List 类型，存储标签的类属性；
4. styles: Map 类型，标签样式；
5. attributes: Map 类型，标签的属性；
6. subHtmlList: List 类型，存储该标签的子标签列表；
7. siblingHtmlList: List 类型，存储该标签的兄弟标签列表；

TextHtml 类只有一个 String 类型的 text 属性。

根据分页数据矩阵生成 HTML 标签的过程如下：

1. 创建 table 标签；
2. 创建 tbody 标签，并将其设置为 table 标签的子标签；
3. 遍历的数据矩阵的每一行，执行 4；
4. 生成 tr 标签，并将其设置为 tbody 标签的子标签；
5. 遍历当前行的每一列，执行 6；
6. 生成 td 标签，并将其设置为当前行对应 tr 标签的子标签；
7. 生成当前单元格数据对应的 TextHtml 对象，并将其设置为 td 标签的数据内容。



容；

5.5.2 EXCEL 文档展示

Apache POI 是一种流行的 API，它允许程序员使用 Java 程序创建、修改和显示 MS Office 文件。它包含类和方法对用户输入数据或文件到 MS Office 文档进行解码。POI 中对应于 EXCEL 操作的核心类如下图所示：

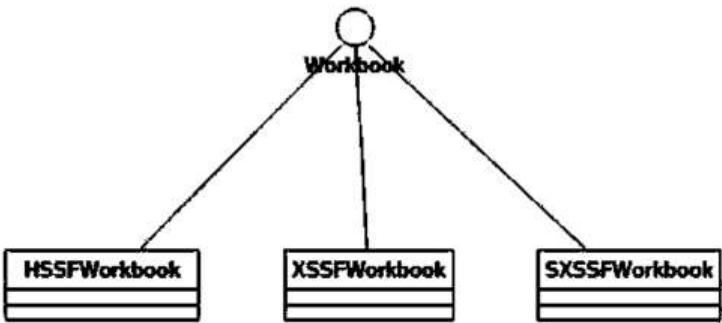


图 5-19 POI 对应于 EXCEL 操作的核心类图

其中，HSSFWorkbook 类支持 03 版本及之前的 Excel 文档操作，即扩展名为.xls 的 Excel 文档；XSSFWorkbook 类支持现有所有版本的 Excel 文档操作，即扩展名为.xlsx 的 Excel 文档；SXSSFWorkbook 类仅支持 07 版本及之后的 Excel 文档写操作。

xls 格式的 Excel 文档和 xlsx 格式的 Excel 文档的区别在于：xls 格式的 Excel 文档中每个 sheet 能够容纳的数据条数为 65536 行，xlsx 格式的 Excel 文档中每个 sheet 能够容纳的数据条数为 1048576 行。

SXSSFWorkbook 和 XSSFWorkbook 的区别在于：XSSFWorkbook 每个 sheet 只能容纳 65536 行数据，SXSSFWorkbook 可以用来处理大数据的导出，使用 XSSFWorkbook 导出会出现内存溢出，但是该类仅支持 xlsx 格式的 Excel 文档的导出，不支持 07 以前的版本，也不支持 Excel 文档的其他操作。

POI 中对数据表操作的核心方法如下：

表 5-1 POI Excel 操作函数一览表

创建工作簿对象	new XSSFWorkbook()
创建工作表对象	XSSFWorkbook.createSheet(String sheetName)
创建行对象	XSSFSheet.createRow(int rowIndex)
创建单元格对象	XSSFRow.createCell(int columnIndex)
写单元格内容	XSSFCell.setCellValue(String cellValue)





基于单元格扩展的报表引擎的研究与实现

创建合并单元格对象	<code>new CellRangeAddress(int fromRow,int fromColumn,int toRow,int toColumn)</code>
添加合并单元格	<code>XSSFSheet.addMergedRegion(CellRangeAddress region)</code>
创建样式对象	<code>XSSFWorkbook.createCellStyle()</code>
设置居中样式	<code>XSSFCellStyle.setAlignment(HSSFCellStyle.ALIGN_CENTER)</code> <code>XSSFCellStyle.setVerticalAlignment(HSSFCellStyle.VERTICAL_CENTER)</code>
设置背景色	<code>XSSFCellStyle.setFillPattern(HSSFCellStyle.SOLID_FOREGROUND);</code> <code>XSSFCellStyle.setFillForegroundColor(HSSFColor.RED.index);</code>
单元格使用样式	<code>XSSFCell.setCellStyle()</code>
创建字体对象	<code>XSSFWorkbook.createFont(XSSFCellStyle style)</code>
设置字体加粗	<code>XSSFFont.setBold(true)</code>
设置字体大小	<code>XSSFFont.setFontHeightInPoints(short fontSize);</code>
使用字体	<code>XSSFCellStyle.setFont(XSSFFont font)</code>



基于单元格扩展的报表引擎的研究与实现



第六章 测试结果及结果分析

本章根据报表设计规则，设计不同应用场景的报表模板，并使用前面章节描述的报表引擎对报表模板进行解析、扩展、填充，最终得到 HTML 页面数据或 EXCEL 文档。

6.1 多源报表

所谓多源报表，就是一张报表中的数据来源于多张数据表甚至多种数据源，这中国式报表的特点之一。

数据库中有两张表，分别为员工表 employee 和部门表 dept，它们的数据如下所示：

empId	empName	salary	dept_id
1	丁丽	4800	4
2	林宏伟	4000	2
3	张伟	5000	2
4	李英杰	3800	2
5	孙帅	4200	3
6	刘斌	7500	3
7	方志浩	5600	4
8	李丹	4500	4
9	周岩	5200	5
10	江山	6500	1

depId	deptName
1	人事部
2	开发部
3	市场部
4	销售部
5	运营部

图 6-1 员工表 employee 和部门表 dept 数据示意图

员工信息表抽象布局如下：

表 6-1 员工信息表抽象定义

	A	B	C	D
1	员工号	员工姓名	员工薪水	员工部门
2	employee.empId	employee.empName	employee.salary	dept.deptName

各单元格意义如下：

表 6-2 员工信息表单元格定义

A1、B1、C1、D1	静态数据单元格、表头标题
A2	数据库单元格，来自 employee 表的 empId 列，纵向扩展
B2	数据库单元格，来自 employee 表的 empName 列，纵向扩展
C2	数据库单元格，来自 employee 表的 salary 列，纵向扩展
D2	数据库单元格，来自 dept 表的 deptName 列，纵向扩展，deptId 等 employee 表中的 deptId 列



基于单元格扩展的报表引擎的研究与实现



员工ID	员工姓名	员工薪水	员工部门
1	丁丽	4800	销售部
2	林宏伟	4000	开发部
3	张伟	5000	开发部
4	李英杰	3800	开发部
5	孙帅	4200	市场部
6	刘斌	7500	市场部
7	方志浩	5600	销售部
8	李丹	4500	销售部
9	周岩	5200	运营部
10	江山	6500	人事部

图 6-2 员工信息表示意图

由上图可见，该报表引擎可以处理多源数据报表。

6.2 条件属性

单元格的`条件属性`是指满足某些条件就会触发的属性事件。

对于上述员工信息表，在单元格 A2 上设置隔行换色条件属性，即如果当前数据在行为偶数行，就给整行数据添加背景色，否则，无背景色。在单元格 C2 上设置高亮条件属性，即如果员工薪水 ≥ 6000 ，则员工薪水列高亮显示。

员工ID	员工姓名	员工薪水	员工部门
1	丁丽	4800	销售部
2	林宏伟	4000	开发部
3	张伟	5000	开发部
4	李英杰	3800	开发部
5	孙帅	4200	市场部
6	刘斌	7500	市场部
7	方志浩	5600	销售部
8	李丹	4500	销售部
9	周岩	5200	运营部
10	江山	6500	人事部

图 6-3 员工信息表条件属性示意图

6.3 分组报表

与列表展示不同，分组报表中相同的数据会合并为一组。

基于单元格扩展的报表引擎的研究与实现



数据库中有班级信息表 classInfo 如下所示：

class	studentName
二班	肖涵
一班	王亚楠
二班	李小乐
一班	刘璐
一班	朱佳璇
一班	刘雯

图 6-4 班级表数据示意图

班级信息表抽象布局如下：

表 6-3 班级信息表抽象定义

	A	B
1	班级	学生姓名
2	classInfo.class	classInfo.studentName

各单元格意义如下：

表 6-4 班级信息表单元格定义

A1、B1	静态数据单元格，表标题
A2	数据库单元格，来自 classInfo 表的 class 列，分组展示
B2	数据库单元格，来自 classInfo 表的 studentName 列，纵向扩展

该报表模板生成的班级信息表如下所示：

班级	学生姓名
一班	王亚楠
	张博
	刘璐
	朱佳璇
	刘雯
二班	王英博
	盖新
	肖涵
	李小乐

图 6-5 班级信息表分组显示示意图

6.4 交叉报表

交叉报表是指报表中的行、列方向都有分组的报表。

数据库中有年级成绩表如下所示：



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

班级	姓名	科目	分数
一班	王亚楠	语文	122
二班	王英博	数学	127
一班	王亚楠	数学	127
二班	王英博	语文	116
一班	王亚楠	物理	84
二班	王英博	物理	86
二班	王英博	生物	88

图 6-6 年级成绩表数据示意图

年级成绩表抽象布局如下：

表 6-5 年级成绩表抽象定义

	A	B	C	D
1	年级成绩表			
2				
3	科目/学生/班级		score.subject	总分
4	socre.class	score.studentName	score.score	SUM(C4)
5	平均分		AVERAGE(C4)	AVERAGE(D4)

各单元格意义如下：

表 6-6 年级成绩表单元格定义

A1、A3、A5	静态单元格，标题
C3	来自数据库中 score 表的 subject 列，横向扩展，分组显示
A4	来自数据库中 score 表的 class 列，纵向扩展，分组显示
B4	来自数据库中 score 表的 studentName 列，纵向扩展，分组显示
C4	来自数据库中 score 表的 score 列，无扩展，汇总显示
D4	公式单元格，对每个学生的分数进行汇总
C5	公式单元格，对每个科目求平均分
D5	公式单元格，对所有学生的总成绩求平均分

该报表模板生成的年级成绩表如下所示：



基于单元格扩展的报表引擎的研究与实现



年级成绩单								
班级	学生	科目						
		语文	数学	物理	生物	化学	英语	总分
一班	王亚楠	122	127	84	93	78	98	602
	张博	110	107	97	67	90	106	577
	刘璐	99	108	85	76	86	127	581
	朱佳璇	108	114	83	90	76	122	593
	刘雯	127	110	79	86	88	127	617
二班	王英博	116	127	86	88	90	125	632
	盖新	107	126	84	82	89	127	615
	肖涵	127	127	88	78	93	112	625
	李小乐	120	127	94	87	83	105	616
平均分		118.11	119.22	86.67	83.00	85.89	116.56	606.4

图 6-7 年级成绩表示意图

由上图可见，该报表引擎可以处理交叉报表。

6.5 按组分页

单元格的按组分页属性是指以单元格的值进行分组，然后分页展示。这种属性常用在分组报表中。

以班级信息表为例，如果想要各个班级的学生信息分页展示，就需要使用到行分页属性。

班级信息表抽象布局如下：

表 6-7 班级信息表抽象定义

	A	B
1	班级	学生姓名
2	classInfo.class	
3		classInfo.studentName

各单元格意义如下：

表 6-8 班级信息表单元格定义

A1、B1	静态数据单元格，表标题，行父格为 A2，设置行前
A2	数据库单元格，来自 classInfo 表的 class 列，纵向扩展，分组展示
B3	数据库单元格，来自 classInfo 表的 studentName 列，纵向扩展

该报表模板生成的班级信息表如下所示：



基于单元格扩展的报表引擎的研究与实现



班级	学生姓名
1001班	王亚楠
	张博
	刘璐
	朱佳璇
	刘雯

图 6-8 班级信息表第一页示意图

班级	学生姓名
1002班	王英博
	盖新
	肖涵
	李小乐

图 6-9 班级信息表第二页示意图

6.6 导出 Excel 文档

由于该报表引擎是基于单元格的，所以天然支持导出 Excel 文档。
带条件属性的员工信息表导出为 Excel 文档如下所示：

	A	B	C	D
1	员工ID	员工姓名	员工薪水	员工部门
2	1	丁丽	4800	销售部
3	2	林宏伟	4000	开发部
4	3	张伟	5000	开发部
5	4	李英杰	3800	开发部
6	5	孙帅	4200	市场部
7	6	刘斌	7500	市场部
8	7	方志浩	5600	销售部
9	8	李丹	4500	销售部
10	9	周岩	5200	运营部
11	10	江山	6500	人事部

图 6-10 年级成绩表导出 Excel 示意图

年级成绩表导出为 Excel 文档如下所示：



基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

第六章 测试结果及结果分析

	A	B	C	D	E	F	G	H	I
	年级成绩单								
1	科目 学生		语文	数学	物理	生物	化学	英语	总分
2									
3	一班	王亚楠	122	127	84	93	78	98	602
4		张博	110	107	97	67	90	106	577
5		刘璐	99	108	85	78	86	127	581
6		朱佳璇	108	114	83	90	76	122	593
7		刘雯	127	110	79	86	88	127	617
8	二班	王英博	116	127	86	88	90	125	632
9		盖新	107	126	84	82	89	127	615
10		肖涵	127	127	88	78	93	112	625
11		李小乐	120	127	94	87	83	105	616
12	平均分		115.11	119.22	86.67	83.00	85.89	116.56	606.44
13									

图 6-11 年级成绩表导出 Excel 示意图



基于单元格扩展的报表引擎的研究与实现



第七章 总结与展望

传统的报表设计器使用了条带式的设计体验，也就是将一张报表分成若干个条区域：标题、页眉、列表头部信息、报表详细信息区域、列表尾部信息、页脚、总等。设计报表模板时先确定使用哪些条状区域，然后在这些条状区域内放置元素。这样的设计虽然简单，但是格式僵化，缺乏变通，而且不能满足复杂场景下中国式报表的要求。

本文的报表设计完全摒弃条带式的设计方式，借鉴了 Excel 的表现形式，采用于单元格的设计方式。

本文描述的报表引擎定义了一种利用 XML 文档来描述报表模板的格式。XML 平台无关性、语言无关性、系统无关性、简单易用性给数据集成与交互带来了极大便利。

此外，本文描述的报表引擎定义了一种单元格扩展原理，基于这种单元格扩展原理对报表中的表格进行扩展、填充，最终得到最终结果矩阵。

最后，该报表引擎支持 HTML 和 Excel 格式的数据展示。

测试结果表明，这种基于单元格扩展原理的报表引擎能够基本满足中国式报表要求。

下一步的研究工作：

1. 数据钻取。数据钻取是在分析时改变维的力度对数据进行层层深入的查看，要是在建立分析时就设定好钻取的维度和层次，在查看时通过鼠标点击某数据点时就会捕捉到下个页面；
2. 树形报表。当我们在实际使用过程中，经常会遇到一种架构为，左边树状|右边显示信息的架构。当点击左侧组织树的时候，右边表格会跟着自动变；
3. 报表设计器。本文的报表模板是根据一定规则设计的 XML 文档，|设计者来说，一个图形化的、直观的报表设计器更为方便；
4. 多种导出格式。应该支持 PDF、WORD、PNG 格式的导出；



基于单元格扩展的报表引擎的研究与实现



参考文献

- [1]陈新林, 张双武. 我国报表系统研究综述[J]. 计算机系统应用, 2007, 16(12):113-117.
- [2]林琴,谭骏珊. 基于 Struts 框架的 Web 报表展示的设计与实现[J]. 计算机系统应用,2006,(11):25-28.
- [3]陈渊, 王泽兵. 企业报表生成系统的研究和实现[J]. 网络新媒体技术, 2000, 21(4):203-206.
- [4]何拥军, 曾文权. 大型企业报表引擎的优化设计与实现[J]. 计算机技术与发展, 23(8): 246-249.
- [5]Li Y, LU T, SHAO X. Study and Implementation of XML-based Web Report Solution[J]. Aeronautical Computer Technique, 2004,34(1):62-65.
- [6]Donoho D L, Johnstone J M. Ideal spatial adaptation by wavelet shrinkage[J]. Biometrika, 1994, 81(3):425-455.
- [7]Eclipse BIRT[EB/OL].<http://eclipse.org/birt>,2017.
- [8]张婧. JasperReport 在 Web 报表中的制作与生成[J]. 信息通信, 2011,(2):134-135.
- [9]Run qian software[EB/OL].<http://www.runqian.com.cn>,2017.
- [10]徐洁, 阳春华, 张祖平. 基于 Java 平台 MVC 模式的流程企业分析检测数据管理系统[J]. 计算机工程与应用, 2005, 41(15):215-217.
- [11]李争鸣, 赵保华. 通用报表生成工具的设计[J]. 计算机研究与发展, 1994, 31(9): 55.
- [12]邵云鹏, 陆志猛. 基于 Web 的企业级报表管理系统的设计与实现[J]. 装备制造技术, 2009,(3):86-88.
- [13]詹黎明.企业级报表自动转换工具的设计与实现[D].东南大学, 2013.
- [14]王佳. 基于 iReport 与 JasperReport 的 Java EE 开源报表解决方案[J]. 湖北第二师范学院学报, 2013, 30(2): 40-41.
- [15]Parra V M, Syed A, Mohammad A, et al. Pentaho and Jaspersoft: A Comparison of Business Intelligence Open Source Tools Processing Big Data to Evaluate Performances[J]. 2016, 7(10):20-29.
- [16]齐白钰, 邢薇, 何鸣. 基于 JAVA 快逸报表工具软件的开发技巧[J].智能计算机应用, 2008, (6):51-52.
- [17]周江. 李勇. 黄鹏. 润乾报表在水库平台信息管理系统中的应用[J]. 企业技术开发

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

- 学术版, 2010, 29(11):11-12.
- [18]朱斌帅, 李蔚, 陈坚红,等. 基于 Web 的热电厂生产过程管理系统的研究与实现
电站系统工程, 2007, 23(6):56-58.
- [19]张浩,蔡晖,刘孟觉,等. 类 Excel 跨平台报表系统[J].自动化技术与应用, 2010, 29(19):22+26.
- [20]David L. Donoho, Iain M. Johnstone. Adapting to Unknown Smoothness via Wavelet Shrinkage[J]. Journal of the American Statistical Association, 1995, 90(432):1200-1211.
- [21]Kaur L, Gupta S, Chauhan R C. Image Denoising Using Wavelet Thresholding[C]//ICVGIP, 2002: 16-18.
- [22]Yergeau F, Bray T, Paoli J, et al. Extensible Markup Language (XML) 1.0[J]. World Wide Web Journal, 2007, 2(4):29-66.
- [23]宋汉增, 沈琳. 利用 Hibernate 对象持久化服务简化 Java 数据库访问[J]. 计算应用, 2003, 23(12): 135-137.
- [24]万东.基于 Struts+Hibernate+Spring 的轻量级 J2EE 框架 J2EE Application Framework Based on Struts+Hibernate+Spring[J]. 现代电子技术, 2011, 34(16):39-41.
- [25]刘芳, 肖铁军. XML 应用的基石: XML 解析技术[J]. 计算机工程与设计, 2005, 26(10): 2823-2824.
- [26]蔚晓娟, 冉静, 李爱华, 等. 基于 DOM 的 XML 解析与应用[J]. 计算机技术与发展, 2007, 17(4): 86-88.
- [27]田斌, 孙霞. 基于 SAX 的 XML 文档解析和存储技术应用[J]. 微机发展, 2005, 15(3): 90-92.
- [28]张迪, 朱敏, 张凌立. 基于 SAX 的 XML 解析与应用[J]. 计算机与数字工程, 2008, 36(7):103-106.
- [29]Zhang M. Application of XML Parser in Wireless Networking[C]// International Conference on Wireless Communications, NETWORKING and Mobile Computing, 2011:1-3.
- [30]张书锋. 基于 Java Excel API 的 excel 文件的操纵技术及其应用[J]. 电脑知识技术: 学术交流, 2010, 6(6): 4565-4566.
- [31]朱文君, 黄国权. 基于 Java 反射机制的 POI 实现 Excel 数据导入/导出[J]. 计算机时代, 2015 (1): 38-39.
- [32]张海波, 董槐林. 一种基于 POI 的 Web 表格生成[J]. 计算机技术与发展, 2008, 18(1): 10-13.

基于单元格扩展的报表引擎的研究与实现



基于单元格扩展的报表引擎的研究与实现

18(2): 21-23.

[33]Li Y, Zuo L, Wei J, et al. Sequential pattern-based cache replacement in servlet container[C]//International Conference on Web Engineering. Springer Berlin Heidelberg 2007: 105-120.

[34]阚薇. 基于 MVC 模式的电信实施管理系统的设计与实现[D]. 吉林大学, 2014.

[35]孙鑫. Servlet/JSP 深入详解:基于 Tomcat 的 Web 开发[M]. 电子工业出版社, 2008,42-56.

君，已阅读到文档结尾了~

立即下载

换一篇

从APP打开该文档，阅读高清版

开通Plus会员，全场文档6折起 >>

相关精品文档

可扩展单元格自定义报表的设计

热度:

单元格地址引用

热度:

Fastreport报表合并单元格技巧

热度:

具有滚动单元格集合的相对阈值感知单元格可视性的自动...

热度:

基于单元格扩展的报表引擎的研究与实现

热度:

热度:

单元格在VBA的表示方法

热度:

DataGridView单元格合并VB实现

热度:

使用分层神经网络的基于视觉的单元格结构识别以及单元...

热度:

热度:

Excel中单元格的引用

热度:

把一个单元格的值直接赋值另一单元格

热度:

基于单元格扩展的报表引擎的研究与实现

